

B 题：城市垃圾分类运输的路径优化与调度

参赛编号： 003193

城市垃圾分类运输的路径优化与调度

摘要

问题 1: 本问采用容量约束车辆路径问题模型 (CVRP)，通过合并具有最大距离节省率的垃圾收集点对，逐步优化车辆调度以降低所需车辆数与总行驶距离。首先，对 30 个垃圾收集点，基于 CVRP 模型求解得到：最优调度方案需使用 **16 辆车**，实现**全局最短路径 1157.23km**。整个算法的时间复杂度为 $O(n^3)$ ，其增长速率在数学上可控，且计算效率可达**毫秒级**，说明在该小规模情境下可行。此外，模型在动态适应性（如收集点较多、车速变动等）方面存在局限，未来可通过混合启发式算法改进优化。

问题 2: 首先，在问题一模型的基础上提出多维度约束扩展方法。相较于问题一，问题二引入了不同类型车辆调度、容积限制、成本限制的约束条件。因此，本文通过对**每种垃圾类型分别进行独立路径计算**，避免混合调度冲突；再根据四种垃圾的平均密度，**取载重容积双约束最小值作实际约束阈值**，确保两类约束限制同时满足。求解结果表明：四类垃圾协同运送的**最短总运行里程为 1110.60km**，**最低总运行成本为 2936.14 元/天**，**总载重 71.40t**。进一步考虑“车辆每日最大行驶时间约束”时，需调整目标函数，嵌入时间约束条件

$$\sum_{i \in N} \sum_{j \in N} t_{i,j,k} \cdot x_{i,j,k} \leq T_{\max,k} \cdot y_k \quad (\forall k \in K)$$

该约束将导致**任务拆分或车辆增派**（如某车辆因 $T_{\max,k}$ 不足需分段运输）。

问题 3: 本问采用 CVRP-熵权 TOPSIS 耦合模型。首先，对 $C_5^0 + \dots + C_5^5 = 32$ 种中转站选择方式，使用 CVRP 模型以邻近原则计算出最优路径，分别计算每种中转站选择下最优路径的距离、成本、载重、碳排放量等信息，并将 CVRP 模型的输出作为后一阶段模型的输入达到耦合目的。其次，采用熵权法确定总成本与碳排放量的权重系数，构建综合评价权重矩阵，通过**适用多指标决策的 TOPSIS 模型**对 32 种中转站选择的综合得分进行排序，筛选出**对称下最优选址方案：在对称条件下选择中转站 32、33、34、35 最优**（成本：3157.19 元/天 | 碳排放量 1281.83kg/天）。在此基础上再**扩展至非对称条件和时间窗口约束**，设计动态绕行策略处理非对称路网，筛选出**非对称下最优选址方案：在非对称条件下选择中转站 33 最优**（成本：3842.00 元/天 | 碳排放量 1431.97kg/天）。结果表明，引入中转站后，日均运输车辆由 16 辆降至 12 辆，实现了降本增效的目的。在对称条件下的算法时间复杂度为 $O(2^k \cdot m \cdot n^2)$ ，在非对称条件下的算法时间复杂度为 $O(2^k \times m \times n^3)$

关键字： CVRP 模型 启发式算法 熵权法 TOPSIS 综合得分

一、问题重述

1.1 问题背景

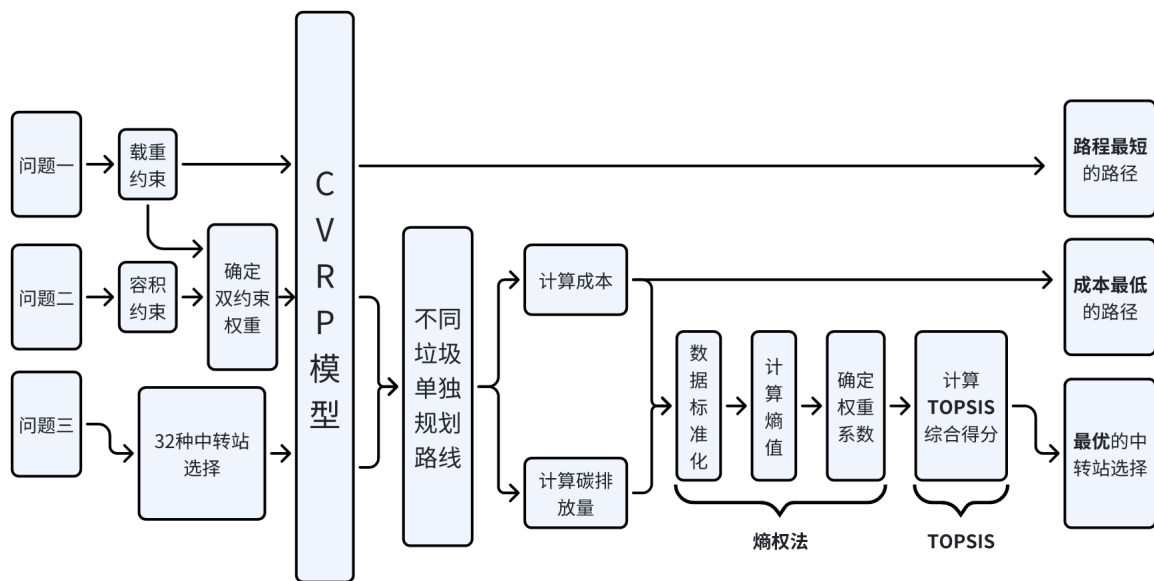
近年来，全球范围内的快速城市化进程加剧了城市固体废弃物的产生与处理压力。作为人口与经济规模领先的国家，我国在垃圾管理领域面临日益严峻的挑战。在垃圾分类政策 [1] 全面实施的背景下，现代垃圾运输体系需应对多维度的复杂约束。不同类别的废弃物（如易腐垃圾、可回收物、危险废物等）因其物理特性和处理要求差异，必须通过专用运输工具进行分类处理。运输车辆在载重能力、容积限制和运营成本等方面存在显著差异，而中转站作为物流网络的关键节点，其存储容量、作业时间窗口及空间分布直接影响整体系统的响应能力。为了解决这一问题，我们尝试通过数学建模，在满足作业规范的前提下实现运输路径的智能规划与车辆的动态调度。这不仅实现了运输成本与低碳目标间的平衡，更为城市环境治理的优化提供了科学依据。

1.2 问题重述

问题一：某城区有 30 个垃圾分类收集点（仅处理“厨余垃圾”），需由专用车辆从固定处理厂出发完成运输任务。已知各收集点坐标、垃圾产生量及车辆最大载重 $Q = 5$ 吨。要求建立数学模型，以最小化每日总行驶距离为目标，确定车辆数量、运输路径及任务分配；针对附件 1 数据设计求解算法，给出最优解（车辆数、路径、里程），并分析算法时间复杂度；讨论模型未考虑现实因素的局限性，提出改进方向。

问题二：垃圾分类扩展为 4 类（厨余、可回收、有害、其他），每类需专用车辆运输，车辆载重不同，且收集点可能产生多类垃圾。要求构建多车辆协同调度模型，目标为最小化总运输成本；同时基于附件 3 和附件 2，扩展问题一算法，考虑载重与容积双约束，求解最优路径与成本；新增“车辆每日最大行驶时间”约束，分析其对路径规划的影响。

问题三：在城区规划若干候选中转站，需考虑中转站容量及碳排放约束。目标为最小化“运输成本 + 中转站建设成本”。要求建立中转站选址-路径优化-碳排放协同模型，明确目标函数与约束条件；设计两阶段算法：第一阶段确定中转站选址与收集点分配，第二阶段优化各中转站车辆路径；最后分析非对称路网对模型复杂度的影响，对比对称与非对称场景下的路径差异。



二、符号说明及名词定义

表 1 文中模型符号汇总

所属问题	符号	说明
问题一：CVRP 模型		
问题一	N_k	第 k 个簇的节点集合（含垃圾处理厂 0）
	K_k	第 k 个簇配备的车辆数
	x_{ij}	决策变量， $x_{ij} = 1$ 表示弧 (i, j) 在最优路径上，否则为 0
	u_i	车辆到达点 i 时的装载量（吨）
	d_{ij}	点 i 到点 j 的行驶距离（km）
	q_i	收集点 i 的垃圾产生量（吨）
	Q	车辆的最大载重限制（吨）
问题二：多约束车辆路径模型		
问题二	K	垃圾类型集合， $K = \{1, 2, 3, 4\}$ （对应 4 类垃圾）
	$x_{k,i,j}$	决策变量，类型 k 车辆从 i 行驶至 j 时取 1，否则为 0
	$y_{k,i}$	决策变量，类型 k 车辆访问收集点 i 时取 1，否则为 0
	$q_{k,i}$	类型 k 车辆到达收集点 i 时的累计载重（吨）
	$v_{k,i}$	类型 k 车辆到达收集点 i 时的累计容积（立方米）

表1（续）文中模型符号汇总

所属问题	符号	说明
	Q_k	类型 k 车辆的载重限制（吨）
	V_k	类型 k 车辆的容积限制（立方米）
	C_k	类型 k 车辆的单位距离运输成本（元/km）
	$w_{i,k}$	收集点 i 产生的类型 k 垃圾重量（吨）
	$v_{i,k}$	收集点 i 产生的类型 k 垃圾容积（立方米）
问题三：中转站选择模型		
	j	中转站组合编号 ($j = 1, 2, \dots, 32$)
	C_j	第 j 个中转站组合的总成本（元）
	E_j	第 j 个中转站组合的碳排放量（千克）
	δ_{jk}	决策变量，选择第 j 个组合的第 k 个中转站时取 1
	D_{jrt}	第 j 个组合中第 t 类车辆在第 r 条路径的行驶距离 (km)
	W_{jrtk}	第 j 个组合中第 t 类车辆运输的第 k 类垃圾重量 (吨)
	S_{jk}	第 j 个组合中第 k 个中转站的处理容量（吨/天）
	Q_r	第 r 辆运输车的载重上限（吨）
熵权-TOPSIS 模型		
综合评价	z_{j1}, z_{j2}	标准化后的总成本和碳排放量指标
	p_{j1}, p_{j2}	归一化后的指标值（用于熵值计算）
	e_1, e_2	指标熵值，反映数据离散程度
	w_1, w_2	熵权法计算的指标权重 ($w_1 + w_2 = 1$)
	V^+, V^-	正负理想解向量（极小型指标：越小越优）
	D_j^+, D_j^-	方案 j 与正负理想解的欧氏距离
	C_j	综合贴近度 ($C_j = \frac{D_j^-}{D_j^+ + D_j^-}$ ，越大越优)

三、 问题一模型的建立与求解

3.1 问题一的模型假设

- 1. 车辆从垃圾处理厂出发，完成所有收集点的运输任务后返回垃圾处理厂，且同一车辆可多次往返（即允许分批运输）。
- 2. 所有运输车辆具有相同的载重容量 $Q = 5$ 吨，且不考虑车辆在运输过程中的损耗及

其他特殊情况。

3. 垃圾收集点的地理位置固定，且每日垃圾产生量服从确定性分布。
4. 车辆在行驶过程中速度恒定为 40km/h，忽略交通拥堵、路况变化等动态因素对行驶时间和路径选择的影响。

3.2 问题一的模型建立

问题一提出车辆需要从垃圾处理厂出发，完成所有运输任务后返回垃圾处理厂，并且允许分批运输。同时，车辆也有最大载重的限制。由此，我们采用**容量约束车辆路径规划模型 (Capacitated Vehicle Routing Problem, CVRP)**。该模型的核心特征与问题一的约束条件高度匹配：

1. 单一中心仓库（垃圾处理厂）作为所有车辆的起点和终点；
2. 有一组具有相同容量的车辆；
3. 目标是在满足车辆容量限制的情况下，设计最优的车辆路线集合。

3.2.1 CVRP 的数学模型

模型构造 对 30 个垃圾收集点 $C_1, C_2, \dots, C_{30}$ ，构建 CVRP 模型，确定最优车辆路径。设每个簇配备 $K_1, K_2, \dots, K_{30}$ 辆车，每辆车容量为 Q

决策变量：

- $x_{ij} \in \{0, 1\}$ ，若弧 (i, j) 被选中则为 1，否则为 0；
- u_i ：车辆到达点 i 时的装载量

目标函数与约束条件：

$$\begin{aligned}
 & \min \sum_{i \in N_k} \sum_{j \in N_k, j \neq i} d_{ij} \cdot x_{ij} \text{ (目标函数)} \\
 s.t. = & \begin{cases} \sum_{j \in N_k, j \neq i} x_{ij} = 1, & \forall i \in C_k & \text{(流量平衡约束 - 点出发)} \\ \sum_{i \in N_k, i \neq j} x_{ij} = 1, & \forall j \in C_k & \text{(流量平衡约束 - 点进入)} \\ \sum_{j \in C_k} x_{0j} \leq K_k & & \text{(车辆数约束)} \\ u_i + q_j \leq u_j + Q(1 - x_{ij}), & \forall i, j \in N_k, i \neq j, i, j \neq 0 & \text{(容量约束 - 运输过程)} \\ q_i \leq u_i \leq Q, & \forall i \in C_k & \text{(容量约束 - 点上装载量)} \\ u_i - u_j + Qx_{ij} \leq Q - q_j, & \forall i, j \in C_k, i \neq j & \text{(子回路消除约束)} \end{cases} \quad (1)
 \end{aligned}$$

3.3 问题一模型的求解与结果分析

对于进行 CVRP 模型分析所采用的具体算法，我们采用了基于节约算法的启发式算法来求得最优解。

3.3.1 基于节约算法的启发式算法流程

Algorithm 1 基于节约算法的启发式算法

收集点坐标、 w_i , $Q = 5$, 处理厂 (原点) 路径集合、总距离、车辆数 K

1. 计算距离矩阵 d_{ij} (含 0 点)。
2. 初始化: $P = \{0 \rightarrow i \rightarrow 0\}_{i=1}^{30}$, $K = 30$ 。
3. 计算 $s(i, j) = d_{0i} + d_{0j} - d_{ij}$, 降序排序所有 (i, j) 。
4. (i, j) 按 s 从大到小 若 i, j 分属不同路径且合并后载重 $\leq Q$, 合并路径, $K--$ 。
5. 输出总距离 P 和车辆数 K 。

3.3.2 结果分析

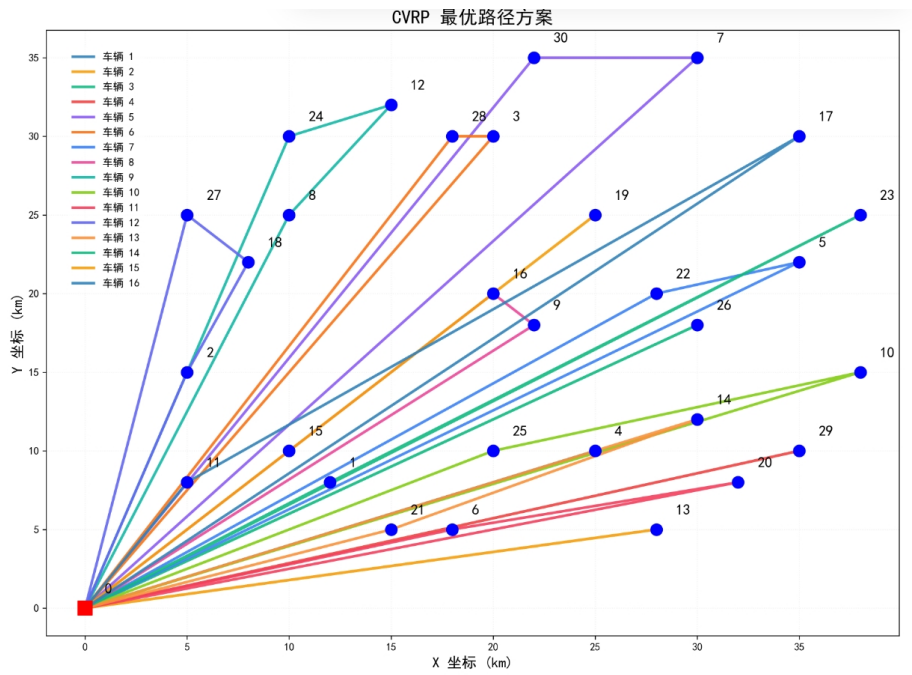


图 1 最优路径示意图 (含收集点标号)

3.3.3 问题一模型的时间复杂度

问题一模型的时间复杂度的计算步骤如下：

在最坏情况下，

1. 生成所有路径对的节约值 $s(i, j)$ 并排序，复杂度为 $O(n^2)$ ；
2. 每次路径合并操作需遍历节点检查载重约束，单次操作最差复杂度为 $O(n)$ ；
3. 总共有 $O(n)$ 次合并操作（从 n 条路径合并为最终路径数）。

表 2 车辆路径、距离及载重量统计

路径	距离 (km)	载重量 (吨)	路径	距离 (km)	载重量 (吨)
0→4→0	53.85	3.10	0→8→12→24→0	72.54	4.60
0→13→0	56.89	3.20	0→10→25→0	81.90	5.00
0→26→0	69.97	3.60	0→6→20→0	65.98	4.90
0→29→0	72.80	3.70	0→2→18→27→0	53.16	4.60
0→7→30→0	95.44	4.80	0→14→21→0	64.68	4.20
0→3→28→0	73.04	4.10	0→1→23→0	90.97	4.70
0→5→22→0	83.03	4.90	0→15→19→0	70.71	4.70
0→9→16→0	59.54	4.90	0→11→17→0	92.73	5.00

使用车辆：16 辆 总行驶距离：1157.23 km 总载重量：70.10 吨

因此，整个算法的时间复杂度为 $O(n^3)$ ，在本问题中收集点的数量 $n = 30$ ，计算时间可控制在 10 毫秒以内，满足实时性的要求。

3.3.4 局限性分析

1. 未考虑交通拥堵、时间窗口（如垃圾处理厂开放时段和收集点允许收集的时段等）和垃圾量波动，导致理论路径与实际执行存在偏差。
2. 未考虑车辆行驶的速度差异，而是认为所有车辆均以 40km/h 的速度匀速行驶，这同样会与实际情况之间产生偏差。
3. 采用的节约算法为启发式算法，无法保证全局最优，可能陷入局部最优（如贪心策略忽略跨路径更优组合）。

3.3.5 改进方向

1. 引入时间窗口约束（VRPTW）

在 CVRP 模型中增加时间窗口约束：

$$\begin{cases} t_i + s_i \leq t_j + M(1 - x_{ij}), & \forall i, j \in N_k, i \neq j \\ a_i \leq t_i \leq b_i, & \forall i \in C_k \end{cases}$$

其中： t_i 为车辆到达点 i 的时间， $[a_i, b_i]$ 为点 i 的时间窗口， M 为足够大的常数， s_i 为点 i 的服务时间。

2. 采用含有速度上升下降过程的函数的车辆速度估算方法

每一轮运输，车辆在每一段路线（从一个垃圾收集点到另一个垃圾收集点）的速度计算公式为：

$$v_i = v_0 \cdot f\left(\frac{x}{d_i}\right)$$

其中 $v_0=40\text{km/h}$ ， d_i 表示每一轮的第 i 段路线的长度， x 为车辆在某一时刻的位置

到它所在段的起点的距离, $f(s)$ 为定义域为 $[0, 1]$ 的非负连续函数, 满足:

(a) $f(0) = f(1) = 0$

(b) $\exists d \in [0, 0.5]$, 使:

- $\forall s \in [0, d], f'(s) > 0$
- $\forall s \in [1 - d, 1], f'(s) < 0$
- $f'(d) = f'(1 - d) = 0$
- $\forall s \in [d, 1 - d], f(s) = 1$

即在每一段路线中, 都有车辆从零加速到 40km/h 以及从 40km/h 减速到零的过程。

3. 采用遗传算法 (GA) 优化

通过交叉、变异操作提升全局寻优能力, 适应度函数设计为:

$$\text{Fit} = \frac{1}{\alpha \cdot L + \beta \cdot V}$$

其中: L 为总行驶距离, V 为载重约束违反度 (如超载量总和), α, β 为权重系数 ($\alpha + \beta = 1$)。

四、 问题二模型的建立与求解

4.1 问题二的模型假设

1. 每类垃圾 (可回收、厨余、有害、其他) 由专用车辆运输, 车辆的载重 Q_k (吨) 和容积 V_k (立方米) 约束独立 ($k = 1, 2, 3, 4$ 表示垃圾类型)。
2. 每类垃圾的密度采用平均值:
 - 厨余垃圾: 1 吨/立方米
 - 可回收物: 0.18 吨/立方米
 - 有害垃圾: 0.8 吨/立方米
 - 其他垃圾: 0.3 吨/立方米
3. 每个收集点 i 的垃圾总量 w_i 可分解为四类垃圾量 $w_{i,k}$ ($w_{i,k} \geq 0$), 且同一收集点的不同类型垃圾由不同车辆处理 (或无该类型垃圾时 $w_{i,k} = 0$)。
4. 车辆仅服务对应类型的垃圾收集点, 路径需从处理厂 (节点 0) 出发, 完成收集后返回处理厂。
5. 忽略车辆启动/刹车能耗、交通拥堵等动态因素, 运输成本与行驶距离线性相关。

4.2 问题二的模型建立

改进的多目标节约算法针对多类型车辆的双约束 (载重 + 容积) 优化。具体步骤如下:

1. **分层路径计算:** 按垃圾类型 k 独立计算路径, 确保不同类型车辆的路径无交叉, 避免混合调度导致的约束冲突。

2. **双约束联合评估:** 在路径合并阶段, 同步检查载重约束 Q_k 和容积约束 V_k : 取两者的最小值作为实际约束阈值 (即 约束阈值 = $\min\left(\frac{\sum w_{m,k}}{Q_k}, \frac{\sum v_{m,k}}{V_k}\right)$), 仅当阈值 ≤ 1 时允许合并。

4.2.1 数学模型

$$\text{决策变量 } x_{k,i,j} = \begin{cases} 1 & \text{类型 } k \text{ 车辆从 } i \text{ 行驶至 } j \\ 0 & \text{否则} \end{cases} \quad (i, j \in N \cup \{0\}).$$

$$y_{k,i} = \begin{cases} 1 & \text{类型 } k \text{ 车辆访问收集点 } i \\ 0 & \text{否则} \end{cases} \quad (i \in N).$$

$q_{k,i}$: 类型 k 车辆到达收集点 i 时的累计载重 (吨)。

$v_{k,i}$: 类型 k 车辆到达收集点 i 时的累计容积 (立方米)。

目标函数与约束条件

$$\begin{aligned} & \min \sum_{k \in K} C_k \sum_{i \in N \cup \{0\}} \sum_{j \in N \cup \{0\}} d_{i,j} x_{k,i,j} \\ \text{s.t. } & \begin{cases} \sum_{k \in K} y_{k,i} \leq 1, \forall i \in N & (\text{类型互斥: 单收集点仅一类车访问}) \\ \sum_j x_{k,0,j} = \sum_j x_{k,j,0}, \forall k \in K & (\text{流量平衡: 车辆出发返回次数相等}) \\ y_{k,i} \leq \frac{w_{i,k} + v_{i,k}}{w_{\min} + v_{\min}}, \forall i, k & (\text{路径覆盖: } w_{\min}, v_{\min} \text{ 为阈值}) \\ q_{k,i} = q_{k,0} + \sum_m w_{m,k} x_{k,m,i}, 0 \leq q_{k,i} \leq Q_k & (\text{载重约束: 累计载重 } Q_k) \\ v_{k,i} = v_{k,0} + \sum_m v_{m,k} x_{k,m,i}, 0 \leq v_{k,i} \leq V_k & (\text{新增: 容积约束: 累计容积 } V_k) \\ \sum_{i,j} \frac{d_{i,j}}{v_k} x_{k,i,j} \leq T_k^{\max}, \forall k \in K & (\text{新增: 时间约束: 总行驶时间 } T_k^{\max}) \\ u_{k,i} - u_{k,j} + M x_{k,i,j} \leq M - w_{j,k}, \forall k, i, j \neq 0 & (\text{子回路消除: MTZ 约束防循环}) \\ x_{k,i,j}, y_{k,i} \in \{0, 1\}, q_{k,i}, v_{k,i} \geq 0 & (\text{变量域: 0-1 变量与非负约束}) \end{cases} \end{aligned}$$

4.3 问题二模型的求解与结果分析

4.3.1 算法流程

Algorithm 2 多类型 CVRP 的改进节约算法

各类型垃圾数据 $w_{i,k}, v_{i,k}$, 车辆参数 Q_k, V_k, C_k , 处理厂坐标 0 各类型车辆路径集合 P_k , 总运输成本 C_{total}

1. 初始化：对每个类型 k , 生成初始路径 $P_k^{(0)} = \{0 \rightarrow i \rightarrow 0\}$ ($\forall i \in N, w_{i,k} > 0$), 记录初始车辆数 $K_k^{(0)} = |\{i \in N \mid w_{i,k} > 0\}|$ 。
2. 计算节约值：对类型 k 的每对收集点 i, j , 计算节约值：

$$s_k(i, j) = d_{0,i} + d_{0,j} - d_{i,j}$$

按 $s_k(i, j)$ 降序排序所有 (i, j) 对。

3. 路径合并：类型 $k \in K$ 按 $s_k(i, j)$ 降序的 (i, j) 对 若 i, j 分属 P_k 中不同路径, 且合并后满足：

$$\max \left(\frac{\sum w_{m,k}}{Q_k}, \frac{\sum v_{m,k}}{V_k} \right) \leq 1$$

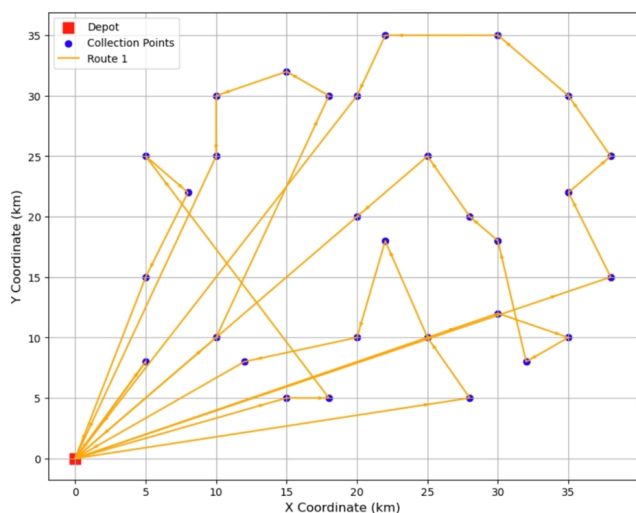
则合并路径 $0 \rightarrow i \rightarrow \dots \rightarrow 0$ 和 $0 \rightarrow j \rightarrow \dots \rightarrow 0$ 为 $0 \rightarrow i \rightarrow j \rightarrow \dots \rightarrow 0$, $K_k \leftarrow K_k - 1$ 。

4. 输出结果：收集各类型车辆的最终路径 P_k , 计算总运输成本 $C_{\text{total}} = \sum_{k \in K} C_k \cdot \sum_{(i,j) \in P_k} d_{i,j}$ 。
-

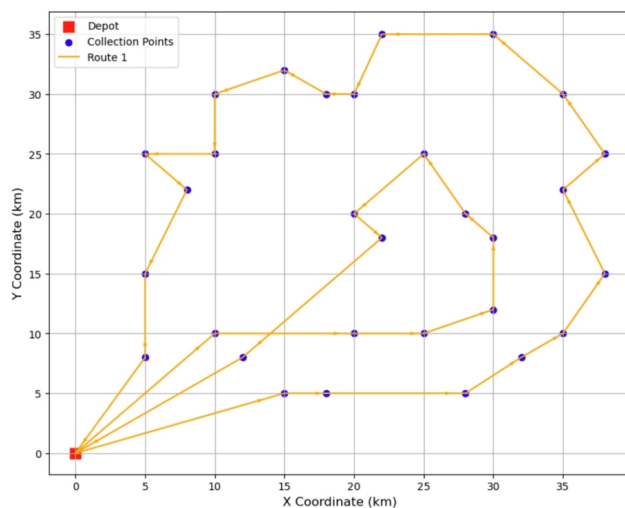
4.3.2 结果分析

表 3 不同类型垃圾运输统计信息

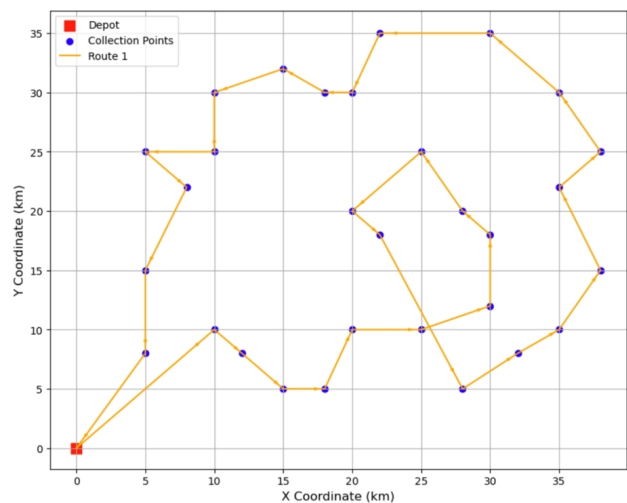
垃圾类型	总距离 (km)	总成本 (yuan)	总重量 (tons)
厨余垃圾	450.10	1125.26	39.39
可回收物	218.33	436.65	8.39
有害垃圾	180.73	903.63	2.31
其他垃圾	261.45	470.60	21.32
总计	1110.60	2936.14	71.40



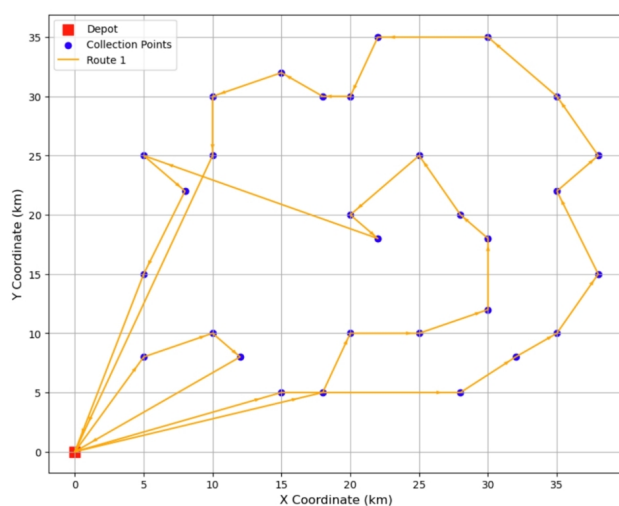
(a) 厨余垃圾运输路径



(b) 可回收垃圾运输路径



(c) 有害垃圾运输路径



(d) 其他垃圾运输路径

图 2 四类垃圾运输路径示意图（详细路径列表见附录表7）

4.4 问题二—引入车辆每日最大行驶时间约束后

4.4.1 CVRP 模型时间约束修改策略

CVRP（容量车辆路径问题）模型需在原有容量约束基础上引入时间约束，具体修改如下：

新增时间参数

- $T_{\max,k}$: 车辆 k 每日最大行驶时间（小时）
- $v = 40 \text{ km/h}$: 车辆行驶速度
- $t_{i,j,k} = \frac{d_{i,j}}{v}$: 车辆 k 从 i 到 j 的行驶时间（小时）
- y_k : 0-1 变量, $y_k = 1$ 表示使用车辆 k

新增时间约束

$$\sum_{i \in N} \sum_{j \in N} t_{i,j,k} \cdot x_{i,j,k} \leq T_{\max,k} \cdot y_k \quad (\forall k \in K)$$

4.4.2 时间约束对路径规划的影响

添加了时间约束之后,原先能一次就运输完毕的路线就有可能因为车辆每日最大行驶时间的约束而分为多轮运输来进行。举例如下:

本题得到的运输有害垃圾的路线原本只要一辆车运输一轮便可以完成,但是如果令 $T_{\max,3} = 3$ (小时),则由于原路线需要 $180.73/40 \approx 4.52$ (小时),超过了 3 小时,所以需要至少分为两轮来进行运输。

五、 问题三的模型建立与求解

5.1 问题三的模型假设

1. 车辆运行假设

车辆行驶速度恒定为 $v_0 = 40 \text{ km/h}$,忽略交通拥堵和临时停车影响;每类垃圾专用车辆不可混运,且单车载重不超过其最大容量 Q_k (k 为垃圾类型)。

2. 垃圾属性假设

各收集点的垃圾产生量为确定性数据,每日固定且无波动;垃圾类型严格分为 4 类(厨余、可回收、有害、其他),每类垃圾的密度已知,容积约束通过载重间接满足。

3. 中转站约束假设

中转站仅作为临时存储节点,无处理能力,其总接收量不超过容量 $C_{j,k}$ (j 为中转站编号, k 为垃圾类型);车辆需在中转站开放时间窗口 $[a_j, b_j]$ 内完成装卸。

4. 成本与排放线性假设

运输成本与行驶距离线性相关,单位距离成本为 c_k (元/km);碳排放量由行驶距离和载重共同决定,单位距离排放系数为 α_k (kg/km),单位载重排放系数为 β_k (kg/吨)。

5.2 问题三的模型建立

中转站选址原则 根据车辆在路径中所到达的垃圾收集点，与中转站的距离远近，选择最近中转站。

5.2.1 32 种中转站选择的最优路径模型

1. 目标函数 (CVRP 扩展模型)

$$\min \begin{cases} \text{总成本 } C_j = \sum_{k=1}^5 T_{jk} \delta_{jk} + \sum_{t=1}^K \sum_{r=1}^{R_t} c_t \cdot D_{jrt} \\ \text{碳排放量 } E_j = \sum_{t=1}^K \sum_{r=1}^{R_t} (\alpha_t D_{jrt} + \beta_t W_{jrt}) \end{cases}$$

($j \in J$: 对每个中转站组合独立计算)

2. 约束条件 (32 种组合下的 CVRP 扩展)

$$\text{s.t.} \begin{cases} \sum_{k=1}^5 \sum_{r=1}^{R_t} W_{jrtk} = Q_{it}, \quad \forall i, t, j \quad (\text{垃圾运输需求约束}) \\ \sum_{r=1}^{R_t} W_{jrtk} \leq S_{jk} \delta_{jk}, \quad \forall j, k, t \quad (\text{中转站容量约束}) \\ a_{jk} \leq \sum_{i=1}^N t_{jrik} \cdot \mathbb{I}(W_{jrtk} > 0) \leq b_{jk}, \quad \forall j, k, r, t \quad (\text{时间窗口约束}) \\ \sum_{t=1}^K \sum_{k=1}^5 W_{jrtk} \leq Q_r, \quad \forall j, r \quad (\text{车辆载重约束}) \\ D_{jri} \neq D_{jir}, \quad \forall j, i, r \quad (\text{非对称路网约束}) \\ \delta_{jk} \in \{0, 1\}, W_{jrtk} \geq 0, \quad \forall j, k, r, t \quad (\text{变量类型约束}) \end{cases}$$

5.2.2 熵权-TOPSIS 综合评价模型

熵权-TOPSIS 模型 [2][3][4] 是一种结合熵权法 (Entropy Weight Method) 与 TOPSIS 法 (Technique for Order Preference by Similarity to Ideal Solution) 的多属性决策方法，用于解决复杂系统中的多目标优化问题。其核心思想是通过客观赋权与理想解逼近，实现对备选方案的科学排序与优选。

1. 熵权法计算指标权重 (输入: 32 组 (C_j, E_j))

$$\left\{ \begin{array}{l} \text{步骤 1: 数据标准化} \\ z_{j1} = \frac{C_j}{\sqrt{\sum_{j=1}^{32} C_j^2}}, \quad z_{j2} = \frac{E_j}{\sqrt{\sum_{j=1}^{32} E_j^2}} \\ \text{步骤 2: 归一化处理} \\ p_{j1} = \frac{z_{j1}}{\sum_{j=1}^{32} z_{j1}}, \quad p_{j2} = \frac{z_{j2}}{\sum_{j=1}^{32} z_{j2}} \\ \text{步骤 3: 计算熵值} \\ e_1 = -\frac{1}{\ln 32} \sum_{j=1}^{32} p_{j1} \ln p_{j1}, \quad e_2 = -\frac{1}{\ln 32} \sum_{j=1}^{32} p_{j2} \ln p_{j2} \\ \text{步骤 4: 确定权重} \\ w_1 = \frac{1 - e_1}{(1 - e_1) + (1 - e_2)}, \quad w_2 = \frac{1 - e_2}{(1 - e_1) + (1 - e_2)} \end{array} \right.$$

2. TOPSIS 综合得分计算 (计算详细列表结果见附录表8和表9)

$$\left\{ \begin{array}{l} \text{步骤 1: 加权标准化矩阵} \\ v_{j1} = w_1 \cdot z_{j1}, \quad v_{j2} = w_2 \cdot z_{j2} \\ \text{步骤 2: 确定正负理想解 (极小型指标)} \\ V^+ = \left(\min_{j=1}^{32} v_{j1}, \min_{j=1}^{32} v_{j2} \right) \\ V^- = \left(\max_{j=1}^{32} v_{j1}, \max_{j=1}^{32} v_{j2} \right) \\ \text{步骤 3: 计算与理想解的距离} \\ D_j^+ = \sqrt{(v_{j1} - V_1^+)^2 + (v_{j2} - V_2^+)^2} \\ D_j^- = \sqrt{(v_{j1} - V_1^-)^2 + (v_{j2} - V_2^-)^2} \\ \text{步骤 4: 综合贴近度 (值越大方案越优)} \\ C_j = \frac{D_j^-}{D_j^+ + D_j^-} \end{array} \right.$$

5.3 问题三模型求解与结果分析

5.3.1 算法流程

Algorithm 3 32 种中转站选址方案评估算法

Require: 收集点坐标 points , 垃圾量 waste_data , 中转站信息 transfer_stations

Ensure: 所有方案的总成本、碳排放及车辆调度计划

- 1: 生成所有中转站组合 $S_{\text{sel}} \subseteq \{1, 2, 3, 4, 5\}$ (共 32 种)
 - 2: **for** 每个组合 S_{sel} **do**
 - 3: 计算建设成本 $C_{\text{con}} = \sum_{j \in S_{\text{sel}}} \gamma_j$
 - 4: **for** 每种垃圾类型 $k \in K$ **do**
 - 5: 计算总垃圾量 $W_k = \sum_{i=1}^{30} w_{i,k}$
 - 6: 计算需车辆数 $N_k = \lceil W_k / Q_k \rceil$
 - 7: 按最近邻策略规划每辆车的路线 (从处理厂出发, 依次访问最近未收集点)
 - 8: 若 $S_{\text{sel}} \neq \emptyset$, 计算到最近中转站的往返距离和额外排放
 - 9: 累计运输成本 C_{trans} 和碳排放 E_{total}
 - 10: **end for**
 - 11: 计算车辆出发/返回时间 (按顺序调度)
 - 12: 保存方案结果 (成本、排放、调度计划)
 - 13: **end for**
 - 14: 输出所有方案结果, 筛选总成本最小的最优方案
-

Algorithm 4 熵权法 + TOPSIS 综合评价算法

Require: 32 种方案的总成本 TC 和碳排放量 E

Ensure: 方案排序结果

- 1: 读取数据并构建指标矩阵 $X = [TC; E]$
 - 2: 极小型指标转换: $X'_{ij} = 1 / (X_{ij} + 10^{-8})$
 - 3: 熵权法计算权重 $w = [w_1, w_2]$
 - 4: TOPSIS 计算贴近度 C_i
 - 5: 按 C_i 降序排序, 输出最优方案
-

5.3.2 结果分析

熵权法计算的指标权重: 总成本权重: 0.2980, 碳排放量权重: 0.7020

表 4 (问题三) 对称条件下 TOPSIS 排序结果 (贴近度越大越优)

方案编号	总成本 (元)	碳排放量 (kg/天)	TOPSIS 得分	排序
31	3157.19	1281.835421	1	1
30	3217.06	3386.932068	0.972818	2
1	3282.47	3498.314683	0.917062	3
...	...	...	...	...
6	8971.72	5824.413559	0.420508	30
5	9630.57	6079.526413	0.402314	31
4	9940.86	6206.943837	0	32

熵权法权重: 总成本 0.4703, 碳排放量 0.5297

表 5 非对称条件下 TOPSIS 综合得分排序（贴近度越大越优）

方案编号	总成本（元）	碳排放量（kg/天）	贴近度	排序
4	3842.001538	1431.978040	0.959814	1
1	3963.996283	1463.789430	0.855949	2
6	4472.959755	1496.935812	0.845908	3
...	...	...	...	...
10	5053.472353	3041.674716	0.163422	30
22	5300.852457	3025.686256	0.065485	31
9	5948.209456	3581.710263	0.000000	32

熵权法权重：总成本 0.2980，碳排放量 0.7020

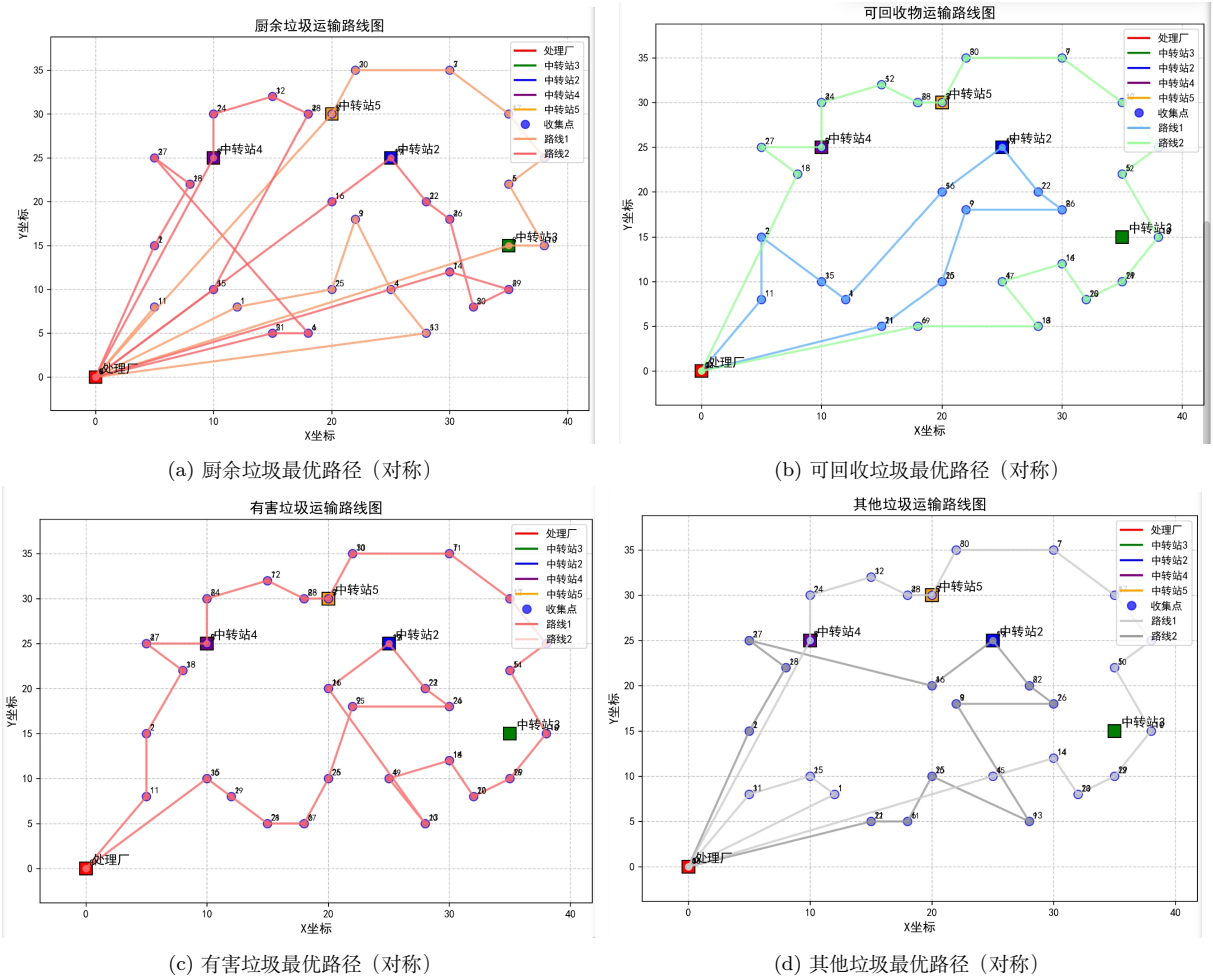
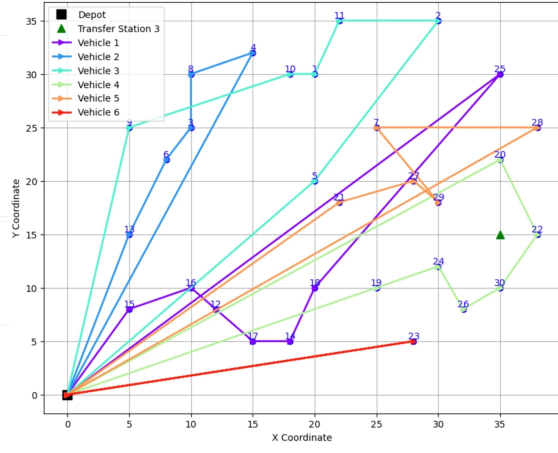
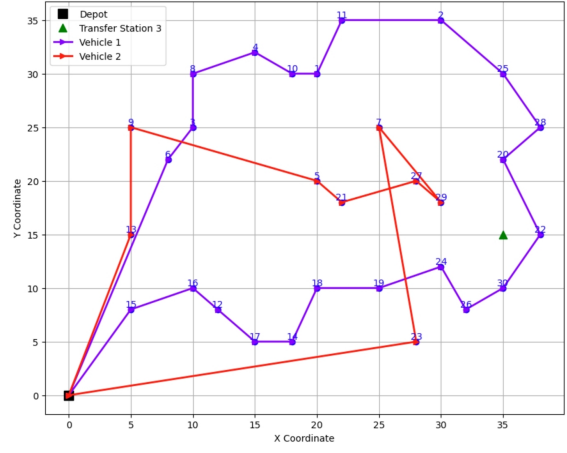


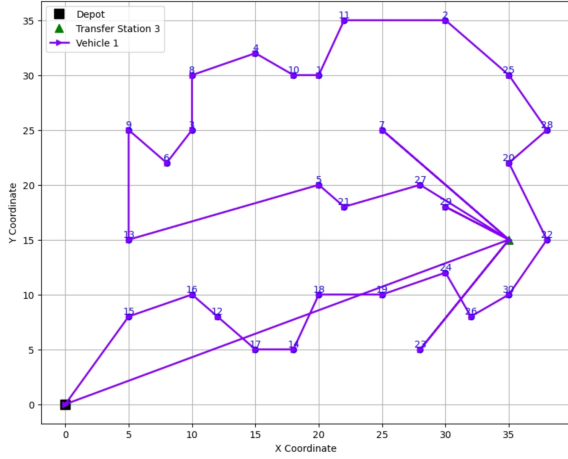
图 3 对称条件下四类垃圾最优路径



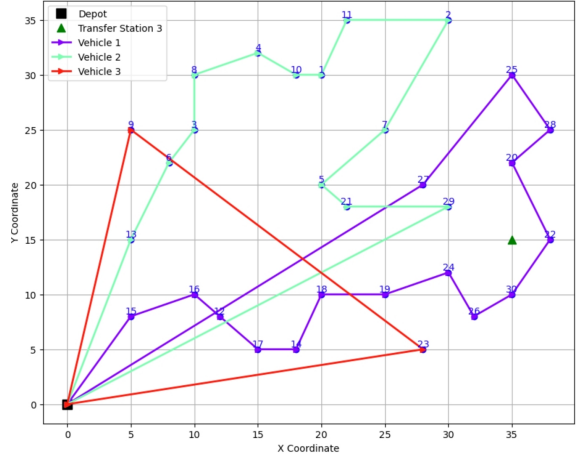
(a) 厨余垃圾最优路径（非对称）



(b) 可回收垃圾最优路径（非对称）



(c) 有害垃圾最优路径（非对称）



(d) 其他垃圾最优路径（非对称）

图 4 非对称条件下四类垃圾最优路径

5.3.3 非对称路网处理

非对称距离矩阵构建 首先,利用欧几里得距离公式构建对称距离矩阵 $D_{\text{sym}} \in \mathbb{R}^{36 \times 36}$ [5] (节点集合 $N = \{0, 1, \dots, 35\}$, 0 为处理厂, 1-30 为收集点, 31-35 为中转站):

$$D_{\text{sym}} = \begin{pmatrix} 0 & d_{0,1} & \cdots & d_{0,35} \\ d_{1,0} & 0 & \cdots & d_{1,35} \\ \vdots & \vdots & \ddots & \vdots \\ d_{35,0} & d_{35,1} & \cdots & 0 \end{pmatrix}, \quad d_{ij} = d_{ji} = \left\lfloor \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \right\rfloor$$

其中, d_{ij} 为节点 i 到 j 的对称距离 (如附件 1 中 $d_{1,2} = 10 \text{ km}$)。

在此基础上,针对附件 5 中的非对称约束 (单行道、禁行时段),对特定节点对的距离值进行修改,得到非对称距离矩阵 $D_{\text{asym}} \in \mathbb{R}^{36 \times 36}$:

$$D = \begin{matrix} & \begin{matrix} 0 & 4 & 9 & 16 & 23 & 27 & 28 & 31 \end{matrix} \\ \begin{matrix} 0 \\ 4 \\ 9 \\ 16 \\ 23 \\ 27 \\ 28 \\ 31 \end{matrix} & \begin{pmatrix} 0 & & & & 40 & & & \\ & 0 & & & & & & 18 \\ & & 0 & 8 & & & & \\ & & & 10 & 0 & & & \\ 45 & & & & & 0 & & \\ & & & & & & 0 & 14 \\ & & & & & & 18 & 0 \\ & & 15 & & & & & 0 \end{pmatrix} \end{matrix}$$

5.3.4 对称与非对称下模型复杂度对比

表 6 对称与非对称条件下时间复杂度对比

情况	时间复杂度表达式	计算公式	具体数量级
对称条件	$O(2^k \cdot m \cdot n^2)$	$2^5 \times 4 \times 30^2 = 115,200$	约 1.15×10^5
非对称条件	$O(2^k \times m \times n^3)$	$2^5 \times 4 \times 30^3 = 3,456,000$	约 3.46×10^6

注： $n = 30$ (垃圾点数量), $k = 5$ (中转站数量), $m = 4$ (垃圾类型数量)

由此观之，在非对称条件下求解的时间复杂度要对称条件下的时间复杂度高一个数量级，但在计算机下的求解速度仍然较为迅速，体现了算法的普适性和优越性。

参考文献

- [1] 国家发展和改革委员会, 住房和城乡建设部. 关于印发“十四五”城镇生活垃圾分类和处理设施建设规划的通知. 政府文件, 2021. https://www.gov.cn/zhengce/zhengceku/2021-05/14/content_606349.htm [访问日期: 2025 年 5 月 25 日].
- [2] 雒玲玉. 基于熵权 TOPSIS 法的钢铁企业财务绩效评价研究. 管理现代化, 2023, 43(3): 112-118. <https://doi.org/10.3969/j.issn.1003-1154.2023.03.024>
- [3] 王兴伟, 李阳, 赵林. 动态需求下车辆路径问题的周期性优化模型及求解. 中国管理科学, 2022, 30(8): 1-12. <http://www.zgglkx.com/CN/10.16381/j.cnki.issn1003-207x.2019.1495>
- [4] 张勇, 陈明明, 李华. 基于改进 TOPSIS 法的公路站场规划布局方案综合评价. 公路交通科技, 2023, 40(5): 150-158. <https://doi.org/10.3969/j.issn.1002-0268.2023.05.018>
- [5] 马海克. 考虑实际路网约束的电动汽车换电站选址研究 [D]. 燕山大学, 2024. DOI:10.27440/d.cnki.gysdu.2024.000499.

附录

A. 数据表格

表 7 （问题二）垃圾路线规划详细列表结果

垃圾类型	路线编号	距离 (km)	重量 (吨)	成本 (元)
厨余垃圾	Route 1	18.87	1.35	47.17
	Route 2	115.05	7.62	287.64
	Route 3	95.51	7.42	238.79
	Route 4	73.73	7.91	184.33
	Route 5	76.60	7.75	191.50
	Route 6	70.33	7.35	175.84
可回收物	Route 1	87.65	2.40	175.30
	Route 2	130.68	5.98	261.35
有害垃圾	Route 1	180.73	2.31	903.63
其他垃圾	Route 1	105.07	9.77	189.12
	Route 2	124.31	9.96	223.76
	Route 3	32.07	1.60	57.73

表 8 （问题三）对称条件下 TOPSIS 排序详细列表结果（贴中度越大越优）

方案序号	成本	碳排放量	TOPSIS 得分	排序
31	3157.19	1281.835421	1	1
30	3217.06	3386.932068	0.972818	2
1	3282.47	3498.314683	0.917062	3
27	3494.26	3575.665047	0.833433	4
28	3721.04	3658.833273	0.815367	5
21	3846.32	3770.215888	0.771878	6
32	4165.13	3825.49028	0.738992	7
20	4221.32	3916.043102	0.71669	8
29	4350.6	3887.98056	0.692252	9
17	4270.9	3928.540572	0.654458	10
26	4459.43	4001.189579	0.651985	11
23	4502.2	4014.223259	0.644285	12
18	4687.68	4076.71354	0.618164	13

表 8 （问题三）对称条件下 TOPSIS 排序详细列表结果（贴近度越大越优）

方案序号	成本	碳排放量	TOPSIS 得分	排序
19	4914.46	4159.881765	0.616383	14
22	5237.4	4289.141158	0.616194	15
7	5464.32	4429.589065	0.615924	16
24	5543.66	4408.700062	0.615655	17
25	5564.4	4404.873819	0.589906	18
14	5451.54	4439.183717	0.562076	19
8	5469.33	4448.28617	0.543479	20
10	5801.25	4561.042363	0.528953	21
12	5901.48	4593.606798	0.525773	22
15	5907.8	4611.579816	0.509874	23
11	6364.64	4781.574849	0.507787	24
9	6656.34	4892.584661	0.50651	25
13	6942.94	4988.0836	0.493271	26
16	7327.96	5139.668801	0.474723	27
3	7763.92	5360.958388	0.456412	28
2	7963.99	5447.525882	0.437827	29
6	8971.72	5824.413559	0.420508	30
5	9630.57	6079.526413	0.402314	31
4	9940.86	6206.943837	0	32
熵权法权重：总成本 0.4703，碳排放量 0.5297				

表 9 （问题三）非对称条件下 TOPSIS 排序详细列表结果（贴近度越大越优）

方案编号	总成本	碳排放量	贴近度	排序
4	3842.001538	1431.978040	0.959814	1
1	3963.996283	1463.789430	0.855949	2
6	4472.959755	1496.935812	0.845908	3
15	4550.153093	1495.091946	0.837599	4
24	4888.099927	1646.096734	0.804947	5
26	5044.140486	1660.925873	0.797258	6
13	5064.398363	1674.719605	0.772094	7
11	5059.636643	1708.412023	0.757068	8

(续表)

(问题三) 非对称条件下 TOPSIS 排序详细列表结果 (贴近度越大越优)

方案编号	总成本	碳排放量	贴近度	排序
3	5114.574326	1761.265438	0.710048	9
31	5549.106486	1783.110637	0.704025	10
25	5827.074450	1976.571795	0.683510	11
23	5815.185740	2039.223735	0.679837	12
12	5816.797536	2113.013946	0.666529	13
16	5802.013235	2135.351206	0.647382	14
14	5878.914502	2164.435309	0.640894	15
5	4511.491097	2255.010618	0.598578	16
32	6474.461901	2249.412076	0.596530	17
2	4563.820970	2376.663614	0.582072	18
30	6506.214676	2298.497037	0.660965	19
20	6319.520411	2317.416692	0.666529	20
17	4405.831705	2496.264485	0.536855	21
21	6412.851400	2358.999959	0.505707	22
28	6560.792595	2410.819761	0.392420	23
27	6640.236210	2425.899823	0.364343	24
19	4681.328523	2720.574948	0.329389	25
29	4985.177475	2721.508285	0.308503	26
8	4784.820049	2870.380250	0.298933	27
18	5014.200829	2849.627979	0.281884	28
7	5003.789836	3011.344484	0.169082	29
10	5053.472353	3041.674716	0.163422	30
22	5300.852457	3025.686256	0.065485	31
9	5948.209456	3581.710263	0.000000	32
熵权法计算的指标权重：总成本权重：0.2980，碳排放量权重：0.7020				

表 10 垃圾运输调度信息（方案 4/32）

垃圾类型	出发时间	返回时间	总距离 (km)	运输总量 (吨)
厨余垃圾	6:00	8:32	181.37	7.982
厨余垃圾	6:00	7:49	72.76	7.305
厨余垃圾	6:00	8:31	101.12	7.944
厨余垃圾	6:00	8:22	95.18	7.698
厨余垃圾	6:00	8:37	104.67	5.880
厨余垃圾	6:00	7:25	56.89	2.580
可回收物	6:00	9:23	135.72	5.987
可回收物	6:00	8:46	110.87	2.399
有害垃圾	6:00	12:33	262.56	2.305
其他垃圾	6:00	8:58	118.87	9.966
其他垃圾	6:00	9:00	120.47	9.089
其他垃圾	6:00	8:06	84.42	2.265

表 11 问题三（不对称节点）运输路线详情（方案 4/32）

垃圾类型	路线及节点详情（起点->(重量,t)->(距离,km)-> 终点）
厨余垃圾	$0 \rightarrow (0,0) \rightarrow 15(5,8) : 1.35 \rightarrow (22.3, km) \rightarrow 16(10,10) : 0.78 \rightarrow (15.6, km) \rightarrow 12(12,8) : 0.72 \rightarrow (8.9, km) \rightarrow 17(15,5) : 0.912 \rightarrow (14.2, km) \rightarrow 14(18,5) : 1.76 \rightarrow (12.5, km) \rightarrow 18(20,10) : 1.74 \rightarrow (19.8, km) \rightarrow 25(35,30) : 0.72 \rightarrow (38.7, km) \rightarrow 0$ 总距离：181.37km，总重量：7.982t
可回收物	$0 \rightarrow (0,0) \rightarrow 13(5,15) : 0.23 \rightarrow (18.6, km) \rightarrow 9(5,25) : 0.34 \rightarrow (10.0, km) \rightarrow 5(20,20) : 0.192 \rightarrow (21.2, km) \rightarrow 21(22,18) : 0.176 \rightarrow (8.3, km) \rightarrow 27(28,20) : 0.195 \rightarrow (14.5, km) \rightarrow 29(30,18) : 0.39 \rightarrow (8.7, km) \rightarrow 7(25,25) : 0.36 \rightarrow (23.1, km) \rightarrow 23(28,5) : 0.516 \rightarrow (20.4, km) \rightarrow 0$ 总距离：110.87km，总重量：2.399t
有害垃圾	$0 \rightarrow (0,0) \rightarrow 15(5,8) : 0.108 \rightarrow (22.3, km) \rightarrow 16(10,10) : 0.065 \rightarrow (15.6, km) \rightarrow 12(12,8) : 0.06 \rightarrow (8.9, km) \rightarrow 17(15,5) : 0.038 \rightarrow (14.2, km) \rightarrow 14(18,5) : 0.096 \rightarrow (12.5, km) \rightarrow 18(20,10) : 0.116 \rightarrow (19.8, km) \rightarrow 25(35,30) : 0.09 \rightarrow (38.7, km) \rightarrow 3(3) : 0 \rightarrow (12.5, km) \rightarrow 0$ 总距离：262.56km，总重量：2.305t（含中转站 3）
其他垃圾	$0 \rightarrow (0,0) \rightarrow 15(5,8) : 0.972 \rightarrow (22.3, km) \rightarrow 16(10,10) : 0.325 \rightarrow (15.6, km) \rightarrow 12(12,8) : 0.3 \rightarrow (8.9, km) \rightarrow 17(15,5) : 0.76 \rightarrow (14.2, km) \rightarrow 14(18,5) : 0.96 \rightarrow (12.5, km) \rightarrow 18(20,10) : 0.725 \rightarrow (19.8, km) \rightarrow 25(35,30) : 0.72 \rightarrow (38.7, km) \rightarrow 0$ 总距离：118.87km，总重量：9.966t

B. 代码

问题一：单车辆调度

```
1 import numpy as np
2     dist = sum(dist_matrix[route[i]][route[i+1]] for i in range(len(route)-1))
3     weight = sum(collection_points[node-1][2] for node in route if node != 0)
4     route_distances.append(dist)
5     route_weights.append(weight)
6 total_distance = sum(route_distances)
7 # 输出结果
8 print(f'总行驶距离: {total_distance:.2f} km')
9 print(f'使用车辆数: {len(final_routes)} 辆')
10 for i, (route, dist, weight) in enumerate(zip(final_routes, route_distances,
11                                             route_weights)):
12     print(f'车辆 {i+1} 路径: {" -> ".join(str(node) for node in route)}, '
13           f'距离: {dist:.2f} km, 载重量: {weight:.2f} 吨')
14 # 可视化设置
15 plt.figure(figsize=(12, 9), dpi=300) # 4:3比例
16 plt.rcParams['font.sans-serif'] = ['SimHei']
17 plt.rcParams['axes.unicode_minus'] = False
18 # 绘制处理厂和收集点
19 x_min, x_max = min(points[:,0]), max(points[:,0])
20 y_min, y_max = min(points[:,1]), max(points[:,1])
21 offset = max(x_max-x_min, y_max-y_min) * 0.025 # 增大偏移量避免遮挡
22 plt.scatter(depot[0], depot[1], c='red', s=180, marker='s', zorder=3)
23 plt.text(depot[0]+offset, depot[1]+offset, '0', fontsize=14, fontweight='bold', zorder=4)
24 for i, point in enumerate(collection_points):
25     plt.scatter(point[0], point[1], c='blue', s=120, zorder=3)
26     plt.text(point[0]+offset, point[1]+offset, str(i+1), fontsize=14, fontweight='bold',
27              zorder=4)
28 colors = [
29     '#3282B8', '#F59E0B', '#10B981', '#EF4444',
30     '#8B5CF6', '#F97316', '#3B82F6', '#EC4899',
31     '#14B8A6', '#84CC16', '#F43F5E', '#6366F1',
32     '#FB923C', '#10B981', '#F59E0B', '#3282B8'
33 ]
34 # 绘制路径并收集图例对象
35 legend_handles = []
36 for i, route in enumerate(final_routes):
37     color = colors[i % len(colors)]
38     coords = [depot] + [collection_points[node-1, :2] for node in route[1:-1]] + [depot]
39     x_route, y_route = zip(*coords)
40     # 绘制路径
41     line, = plt.plot(x_route, y_route, c=color, linewidth=2.5, alpha=0.9, zorder=2)
42     legend_handles.append(line)
43 plt.legend(legend_handles, [f'车辆 {i+1}' for i in range(len(final_routes))],
44           loc='upper left', bbox_to_anchor=(0.02, 0.98), # 左上角边距
```

```

43         frameon=True, framealpha=0.9, edgecolor='white', # 半透明背景
44         fontsize=10, ncol=1, handlelength=2) # 图例条目长度
45 # 图表美化
46 plt.title('CVRP 最优路径方案', fontsize=18, fontweight='bold')
47 plt.xlabel('X 坐标 (km)', fontsize=14)
48 plt.ylabel('Y 坐标 (km)', fontsize=14)
49 plt.grid(alpha=0.15, linestyle=':') # 极淡网格线
50 plt.xlim(x_min - (x_max-x_min)*0.05, x_max + (x_max-x_min)*0.05) # 扩展边界
51 plt.ylim(y_min - (y_max-y_min)*0.05, y_max + (y_max-y_min)*0.05)
52 plt.tight_layout()
53 plt.show()
54 if __name__ == '__main__':
55     solve_cvrp()

```

问题二：多类型约束

```

1  import pandas as pd
2  import numpy as np
3  from itertools import combinations
4  from math import sqrt
5  from datetime import datetime, timedelta
6  # 1. 读取Excel文件
7  file_path = "D:\\四种垃圾.xlsx"
8  data = pd.read_excel(file_path, header=1) # 假设第二行是标题行
9  # 提取坐标和垃圾量数据
10 points = data.iloc[:30, 1:3].values # B2:B32和C2:C32 (30个点)
11 waste_data = data.iloc[:30, 3:7].values # D2:G32 (四种垃圾量)
12 # 2. 垃圾车参数配置
13 vehicle_params = {
14     '厨余垃圾': {'capacity': 8, 'volume': 20, 'distance_cost': 2.5, 'emission1': 0.8,
15                  'emission2': 0.3},
16     '可回收物': {'capacity': 6, 'volume': 25, 'distance_cost': 2, 'emission1': 1.6,
17                  'emission2': 0.5},
18     '有害垃圾': {'capacity': 3, 'volume': 10, 'distance_cost': 5, 'emission1': 0.2,
19                  'emission2': 0.1},
20     '其他垃圾': {'capacity': 10, 'volume': 18, 'distance_cost': 1.8, 'emission1': 1.7,
21                  'emission2': 0.4}
22 }
23 waste_types = list(vehicle_params.keys())
24 # 3. 中转站信息
25 transfer_stations = {
26     1: {'x': 15, 'y': 15, 'cost': 50, 'time': (6, 18), 'capacity': [20, 15, 5, 30]},
27     2: {'x': 25, 'y': 25, 'cost': 60, 'time': (8, 16), 'capacity': [25, 20, 6, 35]},
28     3: {'x': 35, 'y': 15, 'cost': 45, 'time': (10, 14), 'capacity': [18, 12, 4, 25]},
29     4: {'x': 10, 'y': 25, 'cost': 55, 'time': (7, 17), 'capacity': [22, 18, 5, 32]},
30     5: {'x': 20, 'y': 30, 'cost': 58, 'time': (9, 15), 'capacity': [24, 22, 7, 38]}
31 }

```

```

28 # 处理厂位置
29 plant_pos = (0, 0)
30 # 车辆速度 (km/h)
31 speed = 40
32 # 工厂工作时间 (小时)
33 plant_open_time = 6
34 plant_close_time = 18
35 def distance(p1, p2):
36     """计算两点之间的欧几里得距离"""
37     return sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
38 def time_to_str(hours):
39     """将小时数转换为时间字符串"""
40     hours = max(0, min(24, hours))
41     hour = int(hours)
42     minute = int((hours - hour) * 60)
43     return f"{hour:02d}:{minute:02d}"
44 def calculate_route(waste_type_idx, selected_stations):
45     """计算特定垃圾类型的运输路线"""
46     waste_type = waste_types[waste_type_idx]
47     params = vehicle_params[waste_type]
48     capacity = params['capacity']
49     # 获取该类型垃圾的总量
50     total_waste = sum(waste_data[i][waste_type_idx] for i in range(30))
51     if total_waste == 0:
52         return [], 0, 0 # 没有这种垃圾需要运输
53     # 计算需要的车辆数量
54     num_vehicles = int(np.ceil(total_waste / capacity))
55     routes = []
56     total_distance = 0
57     total_emission = 0
58     # 分配垃圾点到车辆
59     remaining_waste = waste_data[:, waste_type_idx].copy()
60     vehicle_loads = [0] * num_vehicles
61     vehicle_routes = [[] for _ in range(num_vehicles)]
62     # 简单分配策略: 按垃圾量从大到小分配给当前负载最少的车辆
63     sorted_points = sorted([(i, remaining_waste[i]) for i in range(30)],
64                             key=lambda x: -x[1])
65     for point_idx, waste in sorted_points:
66         if waste == 0:
67             continue
68         # 找到当前负载最少的车辆
69         min_vehicle = np.argmin(vehicle_loads)
70
71         if vehicle_loads[min_vehicle] + waste <= capacity:
72             vehicle_routes[min_vehicle].append(point_idx)
73             vehicle_loads[min_vehicle] += waste
74             remaining_waste[point_idx] = 0
75     # 为每辆车规划路线

```

```

76 for route in vehicle_routes:
77     if not route:
78         continue
79     # 计算路线距离和碳排放
80     current_pos = plant_pos
81     route_distance = 0
82     route_emission = 0
83     collected_waste = 0
84     # 按照最近邻策略规划路线
85     unvisited = route.copy()
86     while unvisited:
87         # 找到最近的点
88         nearest_idx = min(unvisited, key=lambda i: distance(current_pos, points[i]))
89         nearest_pos = points[nearest_idx]
90         # 计算到该点的距离
91         dist = distance(current_pos, nearest_pos)
92         route_distance += dist
93         route_emission += dist * params['emission1']
94         # 收集垃圾
95         collected_waste += waste_data[nearest_idx][waste_type_idx]
96         route_emission += waste_data[nearest_idx][waste_type_idx] * params['emission2']
97         # 检查是否需要去中转站
98         if selected_stations:
99             # 从transfer_stations字典中获取选中中转站的信息
100             active_stations = {k: transfer_stations[k] for k in selected_stations}
101             # 找到最近的中转站
102             nearest_station = min(active_stations.values(),
103                                   key=lambda s: distance(nearest_pos, (s['x'], s['y'])))
104             station_pos = (nearest_station['x'], nearest_station['y'])
105             # 计算到中转站的距离
106             dist_to_station = distance(nearest_pos, station_pos)
107             route_distance += dist_to_station
108             route_emission += dist_to_station * params['emission1']
109             # 从中转站返回垃圾点（假设需要返回继续收集）
110             route_distance += distance(station_pos, nearest_pos)
111             route_emission += distance(station_pos, nearest_pos) * params['emission1']
112             current_pos = nearest_pos
113         else:
114             current_pos = nearest_pos
115             unvisited.remove(nearest_idx)
116         # 返回处理厂
117         dist_to_plant = distance(current_pos, plant_pos)
118         route_distance += dist_to_plant
119         route_emission += dist_to_plant * params['emission1']
120         total_distance += route_distance
121         total_emission += route_emission
122         routes.append({
123             'waste_type': waste_type,

```

```

124         'distance': route_distance,
125         'emission': route_emission,
126         'load': sum(waste_data[i][waste_type_idx] for i in route)
127     })
128     # 计算运输成本
129     transport_cost = total_distance * params['distance_cost']
130     return routes, transport_cost, total_emission
131 def calculate_schedule(selected_stations):
132     """计算选定中转站后的运输计划和成本"""
133     # 中转站建设成本
134     construction_cost = sum(transfer_stations[s]['cost'] for s in selected_stations)
135     # 计算每种垃圾的运输
136     all_routes = []
137     total_transport_cost = 0
138     total_emission = 0
139     for waste_type_idx in range(4):
140         routes, transport_cost, emission = calculate_route(waste_type_idx, selected_stations)
141         all_routes.extend(routes)
142         total_transport_cost += transport_cost
143         total_emission += emission
144     # 计算车辆出发和返回时间
145     # 按垃圾类型顺序安排车辆
146     current_time = plant_open_time # 6:00
147     for route in all_routes:
148         travel_time = route['distance'] / speed
149         route['departure_time'] = time_to_str(current_time)
150         route['return_time'] = time_to_str(current_time + travel_time)
151         current_time += travel_time
152     # 检查是否超过工厂开放时间
153     if current_time > plant_close_time:
154         print("警告：无法在工厂开放时间内完成所有运输任务")
155     # 总成本 = 建设成本 + 运输成本（转换为万元）
156     total_cost = construction_cost + total_transport_cost / 10000
157     return {
158         'selected_stations': selected_stations,
159         'routes': all_routes,
160         'construction_cost': construction_cost,
161         'transport_cost': total_transport_cost,
162         'total_cost': total_cost,
163         'total_emission': total_emission
164     }
165     # 4. 评估所有32种中转站选址方案
166     all_results = []
167     # 生成所有可能的中转站组合 (2^5 = 32种)
168     for k in range(0, 6):
169         for stations in combinations([1, 2, 3, 4, 5], k):
170             result = calculate_schedule(stations)
171             all_results.append(result)

```

```

172 # 5. 准备输出到Excel的数据
173 output_data = []
174 for result in all_results:
175     output_data.append({
176         '总成本(万元)': result['total_cost'],
177         '碳排放量(千克)': result['total_emission']
178     })
179 # 创建DataFrame并保存到Excel
180 df_output = pd.DataFrame(output_data)
181 output_path = "D:\\总成本和碳排放量.xlsx"
182 df_output.to_excel(output_path, index=False, startrow=1, header=False) # 从A2开始写入
183 # 6. 输出结果到控制台
184 print("32种方案的总成本和碳排放量已保存到:", output_path)
185 # 找出最优方案 (总成本最低)
186 best_result = min(all_results, key=lambda x: x['total_cost'])
187 print("\n最优方案:")
188 print(f"选中站: {best_result['selected_stations']}")
189 print(f"总成本: {best_result['total_cost']:.2f} 万元")
190 print(f"碳排放量: {best_result['total_emission']:.2f} 千克")

```

问题三：CVRP-熵权 TOPSIS 法

1. 生成对称条件下的 32 种方案

```

1  import pandas as pd
2  import numpy as np
3  from itertools import combinations
4  from math import sqrt
5  from datetime import datetime, timedelta
6
7  # 1. 读取Excel文件
8  file_path = "D:\\四种垃圾.xlsx"
9  data = pd.read_excel(file_path, header=1) # 假设第二行是标题行
10
11 # 提取坐标和垃圾量数据
12 points = data.iloc[:30, 1:3].values # B2:B32和C2:C32 (30个点)
13 waste_data = data.iloc[:30, 3:7].values # D2:G32 (四种垃圾量)
14
15 # 2. 垃圾车参数配置
16 vehicle_params = {
17     '厨余垃圾': {'capacity': 8, 'volume': 20, 'distance_cost': 2.5, 'emission1': 0.8,
18                 'emission2': 0.3},
19     '可回收物': {'capacity': 6, 'volume': 25, 'distance_cost': 2, 'emission1': 1.6,
20                 'emission2': 0.5},
21     '有害垃圾': {'capacity': 3, 'volume': 10, 'distance_cost': 5, 'emission1': 0.2,
22                 'emission2': 0.1},
23     '其他垃圾': {'capacity': 10, 'volume': 18, 'distance_cost': 1.8, 'emission1': 1.7,

```

```

        'emission2': 0.4}
21 }
22
23 waste_types = list(vehicle_params.keys())
24
25 # 3. 中转站信息
26 transfer_stations = {
27     1: {'x': 15, 'y': 15, 'cost': 0.013699, 'time': (6, 18), 'capacity': [20, 15, 5, 30]},
28     2: {'x': 25, 'y': 25, 'cost': 0.016438, 'time': (8, 16), 'capacity': [25, 20, 6, 35]},
29     3: {'x': 35, 'y': 15, 'cost': 0.012329, 'time': (10, 14), 'capacity': [18, 12, 4, 25]},
30     4: {'x': 10, 'y': 25, 'cost': 0.015068, 'time': (7, 17), 'capacity': [22, 18, 5, 32]},
31     5: {'x': 20, 'y': 30, 'cost': 0.015890, 'time': (9, 15), 'capacity': [24, 22, 7, 38]}
32 }
33
34 # 处理厂位置
35 plant_pos = (0, 0)
36
37 # 车辆速度 (km/h)
38 speed = 40
39
40 # 工厂工作时间 (小时)
41 plant_open_time = 6
42 plant_close_time = 18
43
44 def distance(p1, p2):
45     """计算两点之间的欧几里得距离"""
46     return sqrt((p1[0]-p2[0])**2 + (p1[1]-p2[1])**2)
47
48 def time_to_str(hours):
49     """将小时数转换为时间字符串"""
50     hours = max(0, min(24, hours))
51     hour = int(hours)
52     minute = int((hours - hour) * 60)
53     return f"{hour:02d}:{minute:02d}"
54
55 def calculate_route(waste_type_idx, selected_stations):
56     """计算特定垃圾类型的运输路线"""
57     waste_type = waste_types[waste_type_idx]
58     params = vehicle_params[waste_type]
59     capacity = params['capacity']
60
61     # 获取该类型垃圾的总量
62     total_waste = sum(waste_data[i][waste_type_idx] for i in range(30))
63     if total_waste == 0:
64         return [], 0, 0 # 没有这种垃圾需要运输
65
66     # 计算需要的车辆数量
67     num_vehicles = int(np.ceil(total_waste / capacity))

```

```

68
69     routes = []
70     total_distance = 0
71     total_emission = 0
72
73     # 分配垃圾点到车辆
74     remaining_waste = waste_data[:, waste_type_idx].copy()
75     vehicle_loads = [0] * num_vehicles
76     vehicle_routes = [[] for _ in range(num_vehicles)]
77
78     # 简单分配策略：按垃圾量从大到小分配给当前负载最少的车辆
79     sorted_points = sorted([(i, remaining_waste[i]) for i in range(30)],
80                             key=lambda x: -x[1])
81
82     for point_idx, waste in sorted_points:
83         if waste == 0:
84             continue
85
86         # 找到当前负载最少的车辆
87         min_vehicle = np.argmin(vehicle_loads)
88
89         if vehicle_loads[min_vehicle] + waste <= capacity:
90             vehicle_routes[min_vehicle].append(point_idx)
91             vehicle_loads[min_vehicle] += waste
92             remaining_waste[point_idx] = 0
93
94     # 为每辆车规划路线
95     for route in vehicle_routes:
96         if not route:
97             continue
98
99         # 计算路线距离和碳排放
100         current_pos = plant_pos
101         route_distance = 0
102         route_emission = 0
103         collected_waste = 0
104
105         # 按照最近邻策略规划路线
106         unvisited = route.copy()
107         while unvisited:
108             # 找到最近的点
109             nearest_idx = min(unvisited, key=lambda i: distance(current_pos, points[i]))
110             nearest_pos = points[nearest_idx]
111
112             # 计算到该点的距离
113             dist = distance(current_pos, nearest_pos)
114             route_distance += dist
115             route_emission += dist * params['emission1']

```



```

116
117     # 收集垃圾
118     collected_waste += waste_data[nearest_idx][waste_type_idx]
119     route_emission += waste_data[nearest_idx][waste_type_idx] * params['emission2']
120
121     # 检查是否需要去中转站
122     if selected_stations:
123         # 从transfer_stations字典中获取选中中转站的信息
124         active_stations = {k: transfer_stations[k] for k in selected_stations}
125         # 找到最近的中转站
126         nearest_station = min(active_stations.values(),
127                               key=lambda s: distance(nearest_pos, (s['x'], s['y'])))
128         station_pos = (nearest_station['x'], nearest_station['y'])
129
130         # 计算到中转站的距离
131         dist_to_station = distance(nearest_pos, station_pos)
132         route_distance += dist_to_station
133         route_emission += dist_to_station * params['emission1']
134
135         # 从中转站返回垃圾点（假设需要返回继续收集）
136         route_distance += distance(station_pos, nearest_pos)
137         route_emission += distance(station_pos, nearest_pos) * params['emission1']
138
139         current_pos = nearest_pos
140     else:
141         current_pos = nearest_pos
142
143     unvisited.remove(nearest_idx)
144
145     # 返回处理厂
146     dist_to_plant = distance(current_pos, plant_pos)
147     route_distance += dist_to_plant
148     route_emission += dist_to_plant * params['emission1']
149
150     total_distance += route_distance
151     total_emission += route_emission
152
153     routes.append({
154         'waste_type': waste_type,
155         'distance': route_distance,
156         'emission': route_emission,
157         'load': sum(waste_data[i][waste_type_idx] for i in route)
158     })
159
160     # 计算运输成本
161     transport_cost = total_distance * params['distance_cost']
162
163     return routes, transport_cost, total_emission

```

```

164
165 def calculate_schedule(selected_stations):
166     """计算选定中转站后的运输计划和成本"""
167     # 中转站建设成本
168     construction_cost = sum(transfer_stations[s]['cost'] for s in selected_stations)
169
170     # 计算每种垃圾的运输
171     all_routes = []
172     total_transport_cost = 0
173     total_emission = 0
174
175     for waste_type_idx in range(4):
176         routes, transport_cost, emission = calculate_route(waste_type_idx, selected_stations)
177         all_routes.extend(routes)
178         total_transport_cost += transport_cost
179         total_emission += emission
180
181     # 计算车辆出发和返回时间
182     # 简单策略：按垃圾类型顺序安排车辆
183     current_time = plant_open_time # 6:00
184
185     for route in all_routes:
186         travel_time = route['distance'] / speed
187         route['departure_time'] = time_to_str(current_time)
188         route['return_time'] = time_to_str(current_time + travel_time)
189
190         current_time += travel_time
191
192     # 检查是否超过工厂开放时间
193     if current_time > plant_close_time:
194         print("警告：无法在工厂开放时间内完成所有运输任务")
195
196     # 总成本 = 建设成本 + 运输成本（转换为万元）
197     total_cost = construction_cost + total_transport_cost / 10000
198
199     return {
200         'selected_stations': selected_stations,
201         'routes': all_routes,
202         'construction_cost': construction_cost,
203         'transport_cost': total_transport_cost,
204         'total_cost': total_cost,
205         'total_emission': total_emission
206     }
207
208 # 4. 评估所有32种中转站选址方案
209 all_results = []
210
211 # 生成所有可能的中转站组合 (2^5 = 32种)

```

```

212 for k in range(0, 6):
213     for stations in combinations([1, 2, 3, 4, 5], k):
214         result = calculate_schedule(stations)
215         all_results.append(result)
216
217 # 5. 输出结果
218 for i, result in enumerate(all_results):
219     print(f"\n方案 {i+1}: 选中转站 {result['selected_stations']}")
220     print(f"建设成本: {result['construction_cost']} 万元")
221     print(f"运输成本: {result['transport_cost']:.2f} 元")
222     print(f"总成本: {result['total_cost']:.2f} 万元")
223     print(f"总碳排放量: {result['total_emission']:.2f} 千克")
224
225     print("\n车辆运输计划:")
226     for j, route in enumerate(result['routes']):
227         print(f" 车辆 {j+1} ({route['waste_type']}):")
228         print(f" 载重: {route['load']:.2f} 吨")
229         print(f" 出发时间: {route['departure_time']}")
230         print(f" 返回时间: {route['return_time']}")
231         print(f" 行驶距离: {route['distance']:.2f} 千米")
232         print(f" 碳排放: {route['emission']:.2f} 千克")
233
234 # 找出最优方案 (总成本最低)
235 best_result = min(all_results, key=lambda x: x['total_cost'])
236 print("\n最优方案:")
237 print(f"选中转站: {best_result['selected_stations']}")
238 print(f"总成本: {best_result['total_cost']:.2f} 万元")
239 print(f"碳排放量: {best_result['total_emission']:.2f} 千克")

```

2. 对称条件下的 32 种方案基于熵权法的 TOPSIS 方法分析

```

1     import numpy as np
2     import pandas as pd
3     from sklearn.preprocessing import MinMaxScaler
4
5     def entropy_weight(data):
6         # 数据标准化
7         scaler = MinMaxScaler()
8         normalized_data = scaler.fit_transform(data)
9
10        # 避免log(0)的情况, 加一个极小值
11        normalized_data = normalized_data + 1e-10
12
13        # 计算熵值
14        m, n = normalized_data.shape
15        p = normalized_data / np.sum(normalized_data, axis=0)
16        e = -1 / np.log(m) * np.sum(p * np.log(p), axis=0)
17

```

```

18     # 计算差异系数和权重
19     d = 1 - e
20     w = d / np.sum(d)
21
22     return w
23
24 def topsis(data, weights):
25     # 数据标准化
26     scaler = MinMaxScaler()
27     normalized_data = scaler.fit_transform(data)
28
29     # 加权标准化决策矩阵
30     weighted_data = normalized_data * weights
31
32     # 确定理想解和负理想解
33     ideal_best = np.max(weighted_data, axis=0)
34     ideal_worst = np.min(weighted_data, axis=0)
35
36     # 计算距离
37     d_best = np.sqrt(np.sum((weighted_data - ideal_best)**2, axis=1))
38     d_worst = np.sqrt(np.sum((weighted_data - ideal_worst)**2, axis=1))
39
40     # 计算相对接近度
41     score = d_worst / (d_worst + d_best)
42
43     return score
44
45 # 读取数据
46 file_path = "D:/总成本和碳排放量.xlsx"
47 data = pd.read_excel(file_path, header=None, usecols=[0, 1], skiprows=1, nrows=32)
48 data.columns = ['总成本', '碳排放量']
49
50 # 提取评价指标数据
51 evaluation_data = data.values
52
53 # 计算熵权法权重
54 weights = entropy_weight(evaluation_data)
55
56 # 应用TOPSIS方法
57 scores = topsis(evaluation_data, weights)
58
59 # 找出最优对象
60 best_index = np.argmax(scores)
61 best_object = best_index + 1 # 因为数据从第2行开始
62
63 # 输出结果
64 print(f"熵权法确定的权重 - 总成本: {weights[0]:.4f}, 碳排放量: {weights[1]:.4f}")
65 print(f"最优对象是第 {best_object} 个对象, TOPSIS得分为: {scores[best_index]:.4f}")

```

```

66
67 # 可选：输出所有对象的得分排名
68 data['TOPSIS得分'] = scores
69 data['排名'] = data['TOPSIS得分'].rank(ascending=False)
70 print("\n所有对象排名:")
71 print(data.sort_values(by='排名'))

```

3. 生成对称条件下最优方案的图像

```

1  import math
2      y.append(transfer_station[1])
3      else:
4          x.append(point_coords[node][0])
5          y.append(point_coords[node][1])
6
7      plt.plot(x, y, 'o-', color=color, linewidth=2, alpha=0.8,
8              label=f'{waste_type}路线{idx+1}' if idx==0 and i==0 else None)
9
10     # 添加图例项
11     if i == 0: # 只添加一次每种类型的图例
12         legend_items.append(Line2D([0], [0], color=color, lw=2))
13         legend_labels.append(waste_type)
14
15     # 添加图例
16     plt.legend(legend_items, legend_labels, loc='upper right', fontsize=10)
17
18     # 添加标题和标签
19     plt.title('各类垃圾运输路线综合图', fontsize=14, fontweight='bold')
20     plt.xlabel('X坐标', fontsize=12)
21     plt.ylabel('Y坐标', fontsize=12)
22     plt.grid(True, linestyle='--', alpha=0.7)
23
24     # 调整坐标轴范围
25     all_x = [depot[0], transfer_station[0]] + [coords[0] for coords in
26         point_coords.values()]
27     all_y = [depot[1], transfer_station[1]] + [coords[1] for coords in
28         point_coords.values()]
29     margin = max(max(all_x) - min(all_x), max(all_y) - min(all_y)) * 0.1
30     plt.xlim(min(all_x) - margin, max(all_x) + margin)
31     plt.ylim(min(all_y) - margin, max(all_y) + margin)
32
33     plt.tight_layout()
34
35     if save_fig and fig_name:
36         plt.savefig(fig_name, dpi=300, bbox_inches='tight')
37
38     plt.show()

```

```

38 # 收集所有类型的路线用于综合可视化
39 all_routes = {
40     "厨余垃圾": [],
41     "可回收物": [],
42     "有害垃圾": [],
43     "其他垃圾": []
44 }
45
46 # 重新运行算法以获取所有路线
47 for k in range(1, 5):
48     waste_type = ["厨余垃圾", "可回收物", "有害垃圾", "其他垃圾"][k-1]
49     Q, alpha, beta, gamma = vehicles[k]
50
51     collect_nodes = []
52     for i in waste:
53         w = waste[i][k-1]
54         if w > 0:
55             collect_nodes.append(i)
56
57     if collect_nodes:
58         routes = clarke_wright_savings(
59             collect_nodes,
60             0,
61             point_distances,
62             {node: waste[node][k-1] for node in collect_nodes},
63             Q,
64             special_points
65         )
66         all_routes[waste_type] = routes
67
68 # 绘制综合路线图
69 visualize_all_routes(
70     all_routes,
71     points,
72     plant,
73     transfer,
74     ["厨余垃圾", "可回收物", "有害垃圾", "其他垃圾"],
75     save_fig=True,
76     fig_name="各类垃圾运输路线综合图.png"
77 )

```

4. 生成非对称条件下的 32 种方案

```

1     import pandas as pd
2     import numpy as np
3     import itertools
4     import math
5     from collections import defaultdict

```

```

6 import time
7
8 # 记录开始时间
9 start_time = time.time()
10
11 # 1. 读取Excel文件
12 file_path = "D:\\四种垃圾.xlsx"
13 data = pd.read_excel(file_path, header=1) # 假设第二行是标题行
14
15 # 提取坐标和垃圾量数据
16 coordinates = data.iloc[:30, 1:3].values # B2:B32和C2:C32 (0-29索引对应30个点)
17 waste_data = data.iloc[:30, 3:7].values # D2:G32 (厨余,可回收,有害,其他)
18
19 # 2. 垃圾车参数
20 vehicle_params = {
21     '厨余垃圾': {'capacity': 8, 'volume': 20, 'cost_per_km': 2.5, 'emission1': 0.8,
22                 'emission2': 0.1},
23     '可回收物': {'capacity': 6, 'volume': 25, 'cost_per_km': 2.0, 'emission1': 1.6,
24                 'emission2': 0.2},
25     '有害垃圾': {'capacity': 3, 'volume': 10, 'cost_per_km': 5.0, 'emission1': 0.2,
26                 'emission2': 0.05},
27     '其他垃圾': {'capacity': 10, 'volume': 18, 'cost_per_km': 1.8, 'emission1': 1.7,
28                 'emission2': 0.15}
29 }
30
31 waste_types = ['厨余垃圾', '可回收物', '有害垃圾', '其他垃圾']
32
33 # 3. 中转站信息
34 transfer_stations = {
35     1: {'x': 15, 'y': 15, 'cost': 500000, 'time': (6, 18),
36         'capacities': {'厨余垃圾': 20, '可回收物': 15, '有害垃圾': 5, '其他垃圾': 30}},
37     2: {'x': 25, 'y': 25, 'cost': 600000, 'time': (8, 16),
38         'capacities': {'厨余垃圾': 25, '可回收物': 20, '有害垃圾': 6, '其他垃圾': 35}},
39     3: {'x': 35, 'y': 15, 'cost': 450000, 'time': (10, 14),
40         'capacities': {'厨余垃圾': 18, '可回收物': 12, '有害垃圾': 4, '其他垃圾': 25}},
41     4: {'x': 10, 'y': 25, 'cost': 550000, 'time': (7, 17),
42         'capacities': {'厨余垃圾': 22, '可回收物': 18, '有害垃圾': 5, '其他垃圾': 32}},
43     5: {'x': 20, 'y': 30, 'cost': 580000, 'time': (9, 15),
44         'capacities': {'厨余垃圾': 24, '可回收物': 22, '有害垃圾': 7, '其他垃圾': 38}}
45 }
46
47 # 垃圾处理厂位置
48 depot = (0, 0)
49
50 # 特殊路径距离规则
51 def get_special_distance(point1, point2, current_time=None):
52     x1, y1 = point1
53     x2, y2 = point2

```

```

50
51 # 检查特殊路径规则
52 if (x1 == 1 and y1 == 12 and x2 == 28 and y2 == 18):
53     return 18 if (x1, y1) < (x2, y2) else 15
54 elif (x1 == 24 and y1 == 10 and x2 == 25 and y2 == 20):
55     return 14 if (x1, y1) < (x2, y2) else 18
56 elif ((x1 == 20 and y1 == 32 and x2 == 0 and y2 == 0) or
57        (x2 == 20 and y2 == 32 and x1 == 0 and y1 == 0)):
58     if current_time is not None and (9 <= current_time < 12):
59         return 45 if (x1, y1) < (x2, y2) else 40
60 elif ((x1 == 6 and y1 == 18 and x2 == 13 and y2 == 28) or
61        (x2 == 6 and y2 == 18 and x1 == 13 and y1 == 28)):
62     if current_time is not None and (9 <= current_time < 11):
63         return 8 if (x1, y1) < (x2, y2) else 10
64
65 # 默认使用欧几里得距离
66 return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
67
68 # 计算两点之间的行驶时间 (小时)
69 def travel_time(point1, point2, current_time=None):
70     distance = get_special_distance(point1, point2, current_time)
71     return distance / 40 # 速度为40 km/h
72
73 # 找到最近的中转站
74 def find_nearest_transfer_station(point, selected_stations):
75     min_dist = float('inf')
76     nearest = None
77     for sid in selected_stations:
78         station = transfer_stations[sid]
79         dist = get_special_distance(point, (station['x'], station['y']))
80         if dist < min_dist:
81             min_dist = dist
82             nearest = sid
83     return nearest
84
85 # 检查中转站是否在开放时间内
86 def is_station_open(station_id, time):
87     station = transfer_stations[station_id]
88     open_time, close_time = station['time']
89     return open_time <= time < close_time
90
91 # 车辆路径规划算法
92 def plan_routes(waste_type, selected_stations):
93     capacity = vehicle_params[waste_type]['capacity']
94     waste_idx = waste_types.index(waste_type)
95
96     # 收集需要服务的点
97     points_to_serve = []

```



```

98     for i in range(30):
99         if waste_data[i][waste_idx] > 0:
100             points_to_serve.append((i, coordinates[i][0], coordinates[i][1],
101                                     waste_data[i][waste_idx]))
102
103     routes = []
104     remaining_points = points_to_serve.copy()
105
106     while remaining_points:
107         # 初始化新路线
108         current_load = 0
109         current_route = []
110         current_time = 6.0 # 最早出发时间
111
112         # 从处理厂出发
113         current_pos = depot
114         route_time = current_time
115         route_distance = 0
116         route_waste = 0
117
118         # 尝试添加点到路线
119         while remaining_points and current_load < capacity:
120             # 找到最近的未服务点
121             min_dist = float('inf')
122             next_point = None
123             next_idx = -1
124
125             for idx, (i, x, y, w) in enumerate(remaining_points):
126                 dist = get_special_distance(current_pos, (x, y), route_time)
127                 if dist < min_dist and current_load + w <= capacity:
128                     min_dist = dist
129                     next_point = (i, x, y, w)
130                     next_idx = idx
131
132             if next_point is None:
133                 break # 没有合适的点了
134
135             # 添加到路线
136             i, x, y, w = next_point
137             travel_dist = get_special_distance(current_pos, (x, y), route_time)
138             travel_t = travel_time(current_pos, (x, y), route_time)
139
140             # 检查是否需要中转站
141             nearest_station = find_nearest_transfer_station((x, y), selected_stations)
142             if nearest_station:
143                 station = transfer_stations[nearest_station]
144                 station_pos = (station['x'], station['y'])

```

```

145         # 计算到中转站的距离和时间
146         to_station_dist = get_special_distance((x, y), station_pos, route_time +
147             travel_t)
148
149         to_station_time = travel_time((x, y), station_pos, route_time + travel_t)
150
151         # 检查中转站是否开放
152         arrival_time = route_time + travel_t + to_station_time
153         if is_station_open(nearest_station, arrival_time):
154             # 可以中转，更新路线
155             current_route.append((i, (x, y), w, True, nearest_station))
156             route_distance += travel_dist + to_station_dist
157             route_time += travel_t + to_station_time
158             current_pos = station_pos
159             current_load += w
160             route_waste += w
161             del remaining_points[next_idx]
162             continue
163
164         # 不中转，直接去下一个点
165         current_route.append((i, (x, y), w, False, None))
166         route_distance += travel_dist
167         route_time += travel_t
168         current_pos = (x, y)
169         current_load += w
170         route_waste += w
171         del remaining_points[next_idx]
172
173         # 返回处理厂
174         return_dist = get_special_distance(current_pos, depot, route_time)
175         return_time = travel_time(current_pos, depot, route_time)
176         route_distance += return_dist
177         route_time += return_time
178
179         # 检查是否能在18:00前返回
180         if route_time > 18:
181             # 无法完成，需要调整（简化处理：跳过）
182             continue
183
184         routes.append({
185             'waste_type': waste_type,
186             'route': current_route,
187             'distance': route_distance,
188             'departure_time': current_time,
189             'return_time': route_time,
190             'total_waste': route_waste
191         })
192
193     return routes

```

```

192
193 # 计算碳排放
194 def calculate_emission(routes):
195     total_emission = 0
196     for route in routes:
197         waste_type = route['waste_type']
198         params = vehicle_params[waste_type]
199         distance = route['distance']
200         waste = route['total_waste']
201
202         emission = distance * params['emission1'] + waste * params['emission2']
203         total_emission += emission
204     return total_emission
205
206 # 计算运输成本
207 def calculate_transport_cost(routes):
208     total_cost = 0
209     for route in routes:
210         waste_type = route['waste_type']
211         cost_per_km = vehicle_params[waste_type]['cost_per_km']
212         total_cost += route['distance'] * cost_per_km
213     return total_cost
214
215 # 格式化时间显示
216 def format_time(decimal_time):
217     hours = int(decimal_time)
218     minutes = int((decimal_time - hours) * 60)
219     return f"{hours}:{minutes:02d}"
220
221 # 主函数：评估所有32种中转站组合
222 def evaluate_all_combinations():
223     station_ids = [1, 2, 3, 4, 5]
224     all_solutions = []
225
226     # 生成所有可能的组合 (2^5 = 32种)
227     for r in range(0, len(station_ids)+1):
228         for selected in itertools.combinations(station_ids, r):
229             selected_stations = list(selected)
230
231             # 计算建设成本
232             construction_cost = sum(transfer_stations[sid]['cost'] for sid in
                                     selected_stations)
233
234             # 为每种垃圾类型规划路线
235             all_routes = []
236             for waste_type in waste_types:
237                 routes = plan_routes(waste_type, selected_stations)
238                 all_routes.extend(routes)

```

```

239
240     # 计算运输成本和碳排放
241     transport_cost = calculate_transport_cost(all_routes)
242     emission = calculate_emission(all_routes)
243
244     total_cost = construction_cost + transport_cost
245
246     solution = {
247         'selected_stations': selected_stations,
248         'construction_cost': construction_cost,
249         'transport_cost': transport_cost,
250         'total_cost': total_cost,
251         'emission': emission,
252         'routes': all_routes
253     }
254     all_solutions.append(solution)
255
256     return all_solutions
257
258 # 运行评估
259 all_solutions = evaluate_all_combinations()
260
261 # 输出所有32种方案
262 for i, solution in enumerate(all_solutions, 1):
263     print(f"\n{'='*50}")
264     print(f"方案 {i}/{len(all_solutions)}")
265     print(f"中转站选择: {solution['selected_stations'] or '无'}")
266     print(f"中转站建设成本: {solution['construction_cost']:,.2f} 元")
267     print(f"运输成本: {solution['transport_cost']:,.2f} 元")
268     print(f"总成本: {solution['total_cost']:,.2f} 元")
269     print(f"碳排放量: {solution['emission']:,.2f} 千克")
270
271     print("\n车辆调度详情:")
272     for route in solution['routes']:
273         print(f"\n垃圾类型: {route['waste_type']}")
274         print(f"出发时间: {route['departure_time']:,.2f} ({format_time(route['departure_time'])})")
275         print(f"返回时间: {route['return_time']:,.2f} ({format_time(route['return_time'])})")
276         print("路线:")
277         for point in route['route']:
278             i, (x, y), w, use_transfer, station_id = point
279             transfer_info = f" -> 中转站{station_id}" if use_transfer else ""
280             print(f"收集点{i+1}({x},{y}) - 重量{w}吨{transfer_info}")
281         print(f"总距离: {route['distance']:,.2f} km")
282         print(f"运输垃圾总量: {route['total_waste']} 吨")
283     print(f"\n{'='*50}")
284
285 # 找出最优方案

```

```

286 best_solution = min(all_solutions, key=lambda x: x['total_cost'])
287 print("\n\n最优方案总结:")
288 print(f"中转站选择: {best_solution['selected_stations']}")
289 print(f"总成本: {best_solution['total_cost']:.2f} 元 (建设:
      {best_solution['construction_cost']:.2f} 元 + 运输:
      {best_solution['transport_cost']:.2f} 元)")
290 print(f"碳排放量: {best_solution['emission']:.2f} 千克")
291
292 # 输出运行时间
293 end_time = time.time()
294 print(f"\n程序运行时间: {end_time - start_time:.2f}秒")

```

5. 非对称条件下的 32 种方案基于熵权法的 TOPSIS 方法分析

```

1     import pandas as pd
2     import numpy as np
3
4     # 1. 读取数据 (跳过标题行, 确保只读取数值)
5     excel_path = "D:\\不对称分析.xlsx"
6     # 标题行在第1行 (Excel的第一行), 数据从第2行开始 (索引1)
7     df = pd.read_excel(excel_path, header=None, skiprows=1, nrows=32, usecols="A:B")
8
9     # 确保数据为纯数值型
10    data = df.values.astype(float) # 若此处报错, 说明数据中仍有非数值内容
11
12    # 2. 数据预处理 (极小型指标转换为极大型)
13    def min_to_max(x):
14        """极小型转极大型函数 (处理一维数组)"""
15        return 1 / (x + 1e-8) # 加极小值避免分母为0
16
17    # 按列应用转换 (axis=0表示按列处理)
18    normalized_data = np.apply_along_axis(min_to_max, axis=0, arr=data)
19
20    # 3. 熵权法计算权重
21    def entropy_weight(data):
22        """计算熵权法权重"""
23        # 归一化: 每个元素除以该列总和
24        normalized = data / data.sum(axis=0)
25
26        # 处理0值 (避免log(0)错误)
27        normalized = np.where(normalized == 0, 1e-8, normalized)
28
29        rows, cols = normalized.shape
30        e = np.zeros(cols)
31
32        for j in range(cols):
33            p = normalized[:, j]
34            e[j] = -np.sum(p * np.log(p)) / np.log(rows)

```

```

35
36     # 计算权重
37     w = (1 - e) / (cols - np.sum(e))
38     return w
39
40 weights = entropy_weight(normalized_data)
41 print("熵权法计算的指标权重: ")
42 print(f"总成本权重: {weights[0]:.4f}, 碳排放量权重: {weights[1]:.4f}")
43
44 # 4. TOPSIS分析
45 def topsis(data, weights, is_mini=True):
46     """TOPSIS算法实现 (支持极小型指标) """
47     # 标准化: 向量规范化
48     std_data = data / np.sqrt((data**2).sum(axis=0))
49
50     # 加权标准化矩阵
51     weighted_data = std_data * weights
52
53     # 确定正负理想解 (极小型指标)
54     if is_mini:
55         z_positive = weighted_data.min(axis=0) # 理想解为最小值
56         z_negative = weighted_data.max(axis=0) # 负理想解为最大值
57     else:
58         z_positive = weighted_data.max(axis=0)
59         z_negative = weighted_data.min(axis=0)
60
61     # 计算到理想解的距离
62     d_positive = np.sqrt(((weighted_data - z_positive)**2).sum(axis=1))
63     d_negative = np.sqrt(((weighted_data - z_negative)**2).sum(axis=1))
64
65     # 计算贴近度 (避免分母为0)
66     c = d_negative / (d_positive + d_negative + 1e-8)
67     return c
68
69 # 执行TOPSIS
70 closeness = topsis(normalized_data, weights, is_mini=True)
71
72 # 5. 结果排序
73 df_result = pd.DataFrame({
74     '方案编号': range(1, 33),
75     '总成本': data[:, 0],
76     '碳排放量': data[:, 1],
77     '贴近度': closeness
78 })
79
80 # 按贴近度降序排序 (贴近度越大越优)
81 df_result = df_result.sort_values(by='贴近度', ascending=False).reset_index(drop=True)
82 df_result['排序'] = df_result.index + 1 # 排序从1开始

```

```

83
84 # 输出结果
85 print("\nTOPSIS排序结果（贴适度越大越优）：")
86 print(df_result[['方案编号', '总成本', '碳排放量', '贴适度', '排序']])
87
88 # 保存结果到Excel
89 result_excel_path = "D:\\不对称分析_排序结果.xlsx"
90 with pd.ExcelWriter(result_excel_path) as writer:
91     df_result.to_excel(writer, sheet_name='排序结果', index=False)
92
93 print(f"\n排序结果已保存至：{result_excel_path}")

```

6. 生成非对称条件下最优方案的图像

```

1     import pandas as pd
2     import numpy as np
3     import itertools
4     import math
5     import time
6     import matplotlib.pyplot as plt
7
8     # 记录开始时间
9     start_time = time.time()
10
11     # 1. 读取Excel文件
12     file_path = "D:\\四种垃圾.xlsx"
13     data = pd.read_excel(file_path, header=1) # 假设第二行是标题行
14
15     # 提取坐标和垃圾量数据
16     coordinates = data.iloc[:30, 1:3].values # B2:B32和C2:C32 (0-29索引对应30个点)
17     waste_data = data.iloc[:30, 3:7].values # D2:G32 (厨余,可回收,有害,其他)
18
19     # 2. 垃圾车参数
20     vehicle_params = {
21         '厨余垃圾': {'capacity': 8, 'volume': 20, 'cost_per_km': 2.5, 'emission1': 0.8,
22                     'emission2': 0.1},
23         '可回收物': {'capacity': 6, 'volume': 25, 'cost_per_km': 2.0, 'emission1': 1.6,
24                     'emission2': 0.2},
25         '有害垃圾': {'capacity': 3, 'volume': 10, 'cost_per_km': 5.0, 'emission1': 0.2,
26                     'emission2': 0.05},
27         '其他垃圾': {'capacity': 10, 'volume': 18, 'cost_per_km': 1.8, 'emission1': 1.7,
28                     'emission2': 0.15}
29     }
30
31     waste_types = ['厨余垃圾', '可回收物', '有害垃圾', '其他垃圾']
32
33     # 3. 中转站信息
34     transfer_stations = {

```

```

31     1: {'x': 15, 'y': 15, 'cost': 136.99, 'time': (6, 18),
32         'capacities': {'厨余垃圾': 20, '可回收物': 15, '有害垃圾': 5, '其他垃圾': 30}},
33     2: {'x': 25, 'y': 25, 'cost': 164.38, 'time': (8, 16),
34         'capacities': {'厨余垃圾': 25, '可回收物': 20, '有害垃圾': 6, '其他垃圾': 35}},
35     3: {'x': 35, 'y': 15, 'cost': 123.29, 'time': (10, 14),
36         'capacities': {'厨余垃圾': 18, '可回收物': 12, '有害垃圾': 4, '其他垃圾': 25}},
37     4: {'x': 10, 'y': 25, 'cost': 150.68, 'time': (7, 17),
38         'capacities': {'厨余垃圾': 22, '可回收物': 18, '有害垃圾': 5, '其他垃圾': 32}},
39     5: {'x': 20, 'y': 30, 'cost': 158.90, 'time': (9, 15),
40         'capacities': {'厨余垃圾': 24, '可回收物': 22, '有害垃圾': 7, '其他垃圾': 38}}
41 }
42
43 # 垃圾处理厂位置
44 depot = (0, 0)
45
46 # 特殊路径距离规则
47 def get_special_distance(point1, point2, current_time=None):
48     x1, y1 = point1
49     x2, y2 = point2
50
51     # 检查特殊路径规则
52     if (x1 == 1 and y1 == 12 and x2 == 28 and y2 == 18):
53         return 18 if (x1, y1) < (x2, y2) else 15
54     elif (x1 == 24 and y1 == 10 and x2 == 25 and y2 == 20):
55         return 14 if (x1, y1) < (x2, y2) else 18
56     elif ((x1 == 20 and y1 == 32 and x2 == 0 and y2 == 0) or
57           (x2 == 20 and y2 == 32 and x1 == 0 and y1 == 0)):
58         if current_time is not None and (9 <= current_time < 12):
59             return 45 if (x1, y1) < (x2, y2) else 40
60     elif ((x1 == 6 and y1 == 18 and x2 == 13 and y2 == 28) or
61           (x2 == 6 and y2 == 18 and x1 == 13 and y1 == 28)):
62         if current_time is not None and (9 <= current_time < 11):
63             return 8 if (x1, y1) < (x2, y2) else 10
64
65     # 默认使用欧几里得距离
66     return math.sqrt((x2 - x1)**2 + (y2 - y1)**2)
67
68 # 计算两点之间的行驶时间 (小时)
69 def travel_time(point1, point2, current_time=None):
70     distance = get_special_distance(point1, point2, current_time)
71     return distance / 40 # 速度为40 km/h
72
73 # 找到最近的中转站
74 def find_nearest_transfer_station(point, selected_stations):
75     min_dist = float('inf')
76     nearest = None
77     for sid in selected_stations:
78         station = transfer_stations[sid]

```



```

79         dist = get_special_distance(point, (station['x'], station['y']))
80         if dist < min_dist:
81             min_dist = dist
82             nearest = sid
83         return nearest
84
85     # 检查中转站是否在开放时间内
86     def is_station_open(station_id, time):
87         station = transfer_stations[station_id]
88         open_time, close_time = station['time']
89         return open_time <= time < close_time
90
91     # 车辆路径规划算法
92     def plan_routes(waste_type, selected_stations):
93         capacity = vehicle_params[waste_type]['capacity']
94         waste_idx = waste_types.index(waste_type)
95
96         # 收集需要服务的点
97         points_to_serve = []
98         for i in range(30):
99             if waste_data[i][waste_idx] > 0:
100                 points_to_serve.append((i, coordinates[i][0], coordinates[i][1],
101                                         waste_data[i][waste_idx]))
102
103         routes = []
104         remaining_points = points_to_serve.copy()
105
106         while remaining_points:
107             # 初始化新路线
108             current_load = 0
109             current_route = []
110             current_time = 6.0 # 最早出发时间
111
112             # 从处理厂出发
113             current_pos = depot
114             route_time = current_time
115             route_distance = 0
116             route_waste = 0
117
118             # 尝试添加点到路线
119             while remaining_points and current_load < capacity:
120                 # 找到最近的未服务点
121                 min_dist = float('inf')
122                 next_point = None
123                 next_idx = -1
124
125                 for idx, (i, x, y, w) in enumerate(remaining_points):
126                     dist = get_special_distance(current_pos, (x, y), route_time)

```

```

126         if dist < min_dist and current_load + w <= capacity:
127             min_dist = dist
128             next_point = (i, x, y, w)
129             next_idx = idx
130
131     if next_point is None:
132         break # 没有合适的点了
133
134     # 添加到路线
135     i, x, y, w = next_point
136     travel_dist = get_special_distance(current_pos, (x, y), route_time)
137     travel_t = travel_time(current_pos, (x, y), route_time)
138
139     # 检查是否需要中转站
140     nearest_station = find_nearest_transfer_station((x, y), selected_stations)
141     if nearest_station:
142         station = transfer_stations[nearest_station]
143         station_pos = (station['x'], station['y'])
144
145         # 计算到中转站的距离和时间
146         to_station_dist = get_special_distance((x, y), station_pos, route_time +
            travel_t)
147         to_station_time = travel_time((x, y), station_pos, route_time + travel_t)
148
149         # 检查中转站是否开放
150         arrival_time = route_time + travel_t + to_station_time
151         if is_station_open(nearest_station, arrival_time):
152             # 可以中转, 更新路线
153             current_route.append((i, (x, y), w, True, nearest_station))
154             route_distance += travel_dist + to_station_dist
155             route_time += travel_t + to_station_time
156             current_pos = station_pos
157             current_load += w
158             route_waste += w
159             del remaining_points[next_idx]
160             continue
161
162         # 不中转, 直接去下一个点
163         current_route.append((i, (x, y), w, False, None))
164         route_distance += travel_dist
165         route_time += travel_t
166         current_pos = (x, y)
167         current_load += w
168         route_waste += w
169         del remaining_points[next_idx]
170
171     # 返回处理厂
172     return_dist = get_special_distance(current_pos, depot, route_time)

```

```

173     return_time = travel_time(current_pos, depot, route_time)
174     route_distance += return_dist
175     route_time += return_time
176
177     # 检查是否能在18:00前返回
178     if route_time > 18:
179         # 无法完成, 需要调整 (简化处理: 跳过)
180         continue
181
182     routes.append({
183         'waste_type': waste_type,
184         'route': current_route,
185         'distance': route_distance,
186         'departure_time': current_time,
187         'return_time': route_time,
188         'total_waste': route_waste
189     })
190
191     return routes
192
193 # 计算碳排放
194 def calculate_emission(routes):
195     total_emission = 0
196     for route in routes:
197         waste_type = route['waste_type']
198         params = vehicle_params[waste_type]
199         distance = route['distance']
200         waste = route['total_waste']
201
202         emission = distance * params['emission1'] + waste * params['emission2']
203         total_emission += emission
204     return total_emission
205
206 # 计算运输成本
207 def calculate_transport_cost(routes):
208     total_cost = 0
209     for route in routes:
210         waste_type = route['waste_type']
211         cost_per_km = vehicle_params[waste_type]['cost_per_km']
212         total_cost += route['distance'] * cost_per_km
213     return total_cost
214
215 # 格式化时间显示
216 def format_time(decimal_time):
217     hours = int(decimal_time)
218     minutes = int((decimal_time - hours) * 60)
219     return f"{hours}:{minutes:02d}"
220

```

```

221 # 绘制路线图
222 def plot_routes(solution, solution_num):
223     # 获取所有路线
224     routes = solution['routes']
225
226     # 按垃圾类型分组
227     routes_by_type = {}
228     for route in routes:
229         waste_type = route['waste_type']
230         if waste_type not in routes_by_type:
231             routes_by_type[waste_type] = []
232         routes_by_type[waste_type].append(route)
233
234     # 为每种垃圾类型创建图表
235     for waste_type, type_routes in routes_by_type.items():
236         plt.figure(figsize=(10, 8))
237
238         # 绘制垃圾处理厂
239         plt.plot(depot[0], depot[1], 'ks', markersize=10, label='Depot')
240
241         # 绘制收集点
242         for i in range(30):
243             if waste_data[i][waste_types.index(waste_type)] > 0:
244                 plt.plot(coordinates[i][0], coordinates[i][1], 'bo')
245                 plt.text(coordinates[i][0], coordinates[i][1], f'{i+1}',
246                         ha='center', va='bottom', color='blue')
247
248         # 绘制中转站
249         for sid in solution['selected_stations']:
250             station = transfer_stations[sid]
251             plt.plot(station['x'], station['y'], 'g^', markersize=8, label=f'Transfer
                Station {sid}')
252
253         # 绘制路线
254         colors = plt.cm.rainbow(np.linspace(0, 1, len(type_routes)))
255         for route_idx, route in enumerate(type_routes):
256             color = colors[route_idx]
257             path = [depot] # 从处理厂出发
258
259             for point in route['route']:
260                 i, (x, y), w, use_transfer, station_id = point
261                 path.append((x, y))
262                 if use_transfer:
263                     station = transfer_stations[station_id]
264                     path.append((station['x'], station['y']))
265
266             path.append(depot) # 返回处理厂
267

```

```

268         # 绘制路径
269         x_coords = [p[0] for p in path]
270         y_coords = [p[1] for p in path]
271         plt.plot(x_coords, y_coords, '->', color=color, linewidth=2,
272                 label=f'Vehicle {route_idx+1}')
273
274     plt.title(f'Solution {solution_num} - {waste_type} Collection Routes')
275     plt.xlabel('X Coordinate')
276     plt.ylabel('Y Coordinate')
277     plt.grid(True)
278
279     # 处理图例重复问题
280     handles, labels = plt.gca().get_legend_handles_labels()
281     by_label = dict(zip(labels, handles))
282     plt.legend(by_label.values(), by_label.keys(), loc='best')
283
284     plt.show()
285
286 # 主函数：评估所有32种中转站组合
287 def evaluate_all_combinations():
288     station_ids = [1, 2, 3, 4, 5]
289     all_solutions = []
290
291     # 生成所有可能的组合 (2^5 = 32种)
292     for r in range(0, len(station_ids)+1):
293         for selected in itertools.combinations(station_ids, r):
294             selected_stations = list(selected)
295
296             # 计算建设成本
297             construction_cost = sum(transfer_stations[sid]['cost'] for sid in
298                                     selected_stations)
299
300             # 为每种垃圾类型规划路线
301             all_routes = []
302             for waste_type in waste_types:
303                 routes = plan_routes(waste_type, selected_stations)
304                 all_routes.extend(routes)
305
306             # 计算运输成本和碳排放
307             transport_cost = calculate_transport_cost(all_routes)
308             emission = calculate_emission(all_routes)
309
310             total_cost = construction_cost + transport_cost
311
312             solution = {
313                 'selected_stations': selected_stations,
314                 'construction_cost': construction_cost,

```

```

315         'total_cost': total_cost,
316         'emission': emission,
317         'routes': all_routes
318     }
319     all_solutions.append(solution)
320
321     return all_solutions
322
323 # 运行评估
324 all_solutions = evaluate_all_combinations()
325
326 # 输出方案4的路线图
327 solution_4 = all_solutions[3] # 方案4是索引3
328 print(f"\nPlotting routes for Solution 4 (中转站选择: {solution_4['selected_stations']})")
329 plot_routes(solution_4, 4)
330
331 # 输出所有32种方案
332 for i, solution in enumerate(all_solutions, 1):
333     print(f"\n{'='*80}")
334     print(f"方案 {i}/{len(all_solutions)}")
335     print(f"中转站选择: {solution['selected_stations'] or '无'}")
336     print(f"中转站建设成本: {solution['construction_cost']:.2f} 元")
337     print(f"运输成本: {solution['transport_cost']:.2f} 元")
338     print(f"总成本: {solution['total_cost']:.2f} 元")
339     print(f"碳排放量: {solution['emission']:.2f} 千克")
340
341     print("\n车辆调度详情:")
342     for route in solution['routes']:
343         print(f"\n垃圾类型: {route['waste_type']}")
344         print(f"出发时间: {route['departure_time']:.2f} ({format_time(route['departure_time'])})")
345         print(f"返回时间: {route['return_time']:.2f} ({format_time(route['return_time'])})")
346         print("路线:")
347         for point in route['route']:
348             i, (x, y), w, use_transfer, station_id = point
349             transfer_info = f" -> 中转站{station_id}" if use_transfer else ""
350             print(f"收集点{i+1}({x},{y}) - 重量{w}吨{transfer_info}")
351         print(f"总距离: {route['distance']:.2f} km")
352         print(f"运输垃圾总量: {route['total_waste']} 吨")
353     print(f"\n{'='*80}")
354
355 # 找出最优方案
356 best_solution = min(all_solutions, key=lambda x: x['total_cost'])
357 print("\n\n最优方案总结:")
358 print(f"中转站选择: {best_solution['selected_stations']}")
359 print(f"总成本: {best_solution['total_cost']:.2f} 元 (建设:
    {best_solution['construction_cost']:.2f} 元 + 运输:
    {best_solution['transport_cost']:.2f} 元)")

```

```
360 print(f"碳排放量: {best_solution['emission']:.2f} 千克")
361
362 # 输出运行时间
363 end_time = time.time()
364 print(f"\n程序运行时间: {end_time - start_time:.2f}秒")
```